



High Education and Research Ministry
Hassiba Benbouali University of Chlef
Faculty of Exact Sciences and Informatics
Computer Science Department



END OF STUDIES PROJECT MEMORANDUM

MASTER IN COMPUTER SCIENCE

SPECIALTY:

SOFTWARE ENGINEERING

By

MOUNNAH YOUNES

HAMDANI MOHAMED AMINE

Theme :

**Improving YOLO Accuracy for Object Detection in
Complex Scenes**

Supported on 10/06/2025 Before the jury:

Ms Abboura Asma

Grade/ UHB-Chlef

President

Ms Bousahba Nassima

Grade/ UHB-Chlef

Examiner

Mr Medjahed Chahreddine

Grade/ UHB-Chlef

Supervisor

Academic year 2024/2025

Acknowledgements

We would like to express our sincere thanks to all the people who contributed, directly or indirectly, to the completion of this work.

Our deepest gratitude goes especially to Mr. Medjahed, our supervisor, for his availability, remarks, and advice throughout each phase of our project.

We would also like to express our appreciation to Ms. Krolkral, our co-supervisor, for her valuable guidance, continuous support, and constructive feedback during the development of this work.

We also thank Ms. Abboura and Ms. Bousahba, members of the jury, who kindly agreed to evaluate our work.

Dedication

To our dear parents who spared no effort in supporting and accompanying us throughout our academic journey,
To our beloved sisters and brothers for their constant support,
To our friends and loved ones, we dedicate this humble work to express our gratitude to them.

Abstract

Computer vision relies on object detection and supports different applications, including self-driving cars and intelligent monitoring. YOLOv8 has greatly helped by improving accuracy and making the inference process faster. Even so, it is still difficult to achieve good real-time performance when conditions include clutter, obstructions, and objects that are small in size.

This thesis introduces some new improvements to the YOLOv8 architecture that involve C3TR, ADown, and the use of several sizes of receptive fields. On top of that, TensorRT and OpenVINO optimization methods were applied to retain real-time operation regardless of the hardware used.

Results from experiments reveal that the improved YOLOv8 performs better than its baseline in terms of both speed and accuracy across various datasets. Thanks to these findings, future studies can be conducted on better, more reliable, and more accurate object detection techniques.

Résumé

La vision par ordinateur repose sur la détection d'objets et prend en charge diverses applications, notamment les voitures autonomes et la surveillance intelligente. YOLOv8 a grandement contribué à améliorer la précision et à accélérer le processus d'inférence. Malgré cela, il reste difficile d'obtenir de bonnes performances en temps réel en présence d'encombrement, d'obstructions et d'objets de petite taille.

Cette thèse présente de nouvelles améliorations de l'architecture YOLOv8, notamment C3TR, ADown et l'utilisation de champs récepteurs de différentes tailles. De plus, les méthodes d'optimisation TensorRT et OpenVINO ont été appliquées pour garantir un fonctionnement en temps réel quel que soit le matériel utilisé.

Les résultats expérimentaux révèlent que la version améliorée de YOLOv8 offre de meilleures performances que la version de référence en termes de vitesse et de précision sur divers jeux de données. Grâce à ces résultats, de futures études pourront être menées sur des techniques de détection d'objets plus performantes, plus fiables et plus précises.

Contents

General Introduction	1
1 Introduction	2
1.1 Introduction	3
1.2 Background and Motivation	3
1.2.1 Overview of Object Detection in Computer Vision	3
1.2.2 Importance of Real-Time Detection in Complex Scenes	3
1.2.3 Motivation for Choosing YOLO as the Baseline Model	4
1.3 Problem Statement	4
1.3.1 Challenges Faced by YOLO in Complex Scenes	4
1.3.2 Limitations in Detecting Small, Overlapping, or Occluded Objects	5
1.4 Artificial Intelligence	5
1.4.1 Domains of Artificial Intelligence	6
1.4.2 Specialized Areas of Artificial Intelligence	8
1.5 Object Detection and Tracking	11
1.5.1 Object Detection	11
1.5.2 Object Tracking	13
1.6 Research Objectives	14
1.7 Research Questions and Hypotheses	15
1.8 Contributions of the Study	16
1.9 Conclusion	16
2 Literature Review	17
2.1 Introduction	18
2.2 Overview of YOLO Architecture	18
2.2.1 YOLO Evolution: YOLOv1 to YOLOv8	18
2.2.2 How YOLO Works	19
2.2.3 YOLOv8 Architecture	21
2.3 Performance Analysis	24
2.3.1 Accuracy and Speed Trade-offs	24

2.3.2	Comparison with YOLOv7 and Other Object Detection Models	25
2.3.3	Evaluation Metrics (mAP, FPS, Latency)	25
2.4	Challenges in Complex Scene Detection	28
2.4.1	Core Obstacles in Complex Scene Detection	28
2.4.2	Real-world Dataset Examples	29
2.5	Related Work on Improving YOLO Accuracy	29
2.5.1	Model Architecture Enhancements	29
2.5.2	Integration of Attention Mechanisms	29
2.5.3	Ensemble Learning and Post-processing Methods	30
2.6	State of the Art	30
2.7	Summary and Research Gaps	31
2.8	Conclusion	32
3	Methodology	33
3.1	Introduction	34
3.2	Justification for Using YOLOv8	34
3.3	Proposed Improvements to YOLO	35
3.3.1	Architectural Modifications for Enhanced Accuracy	35
3.3.2	Speed Optimization Strategy	39
3.3.3	Code Architecture	41
3.4	Conclusion	43
4	Experimental Results and Analysis	44
4.1	Introduction	45
4.2	Objectives	45
4.3	Implementation Details	46
4.3.1	Tools and Libraries	46
4.3.2	Hardware Configuration	48
4.4	Overall Research Framework	48
4.5	Dataset Selection	49
4.6	Baseline Performance Evaluation	50
4.7	Performance of the Improved YOLO Model	51
4.8	Comparison Between Original and Improved YOLOv8n Model	52
4.9	Visual Comparison Between YOLOv8 and Improved Model	53
4.10	YOLOv8 and Improved YOLOv8 Comparisons Before and After Augmentation	54
4.10.1	Data Augmentation and Its Benefits	54
4.10.2	Visual Comparison Between YOLOv8 and Improved YOLOv8 Before and After Augmentation	55

4.11 Visual Comparison of YOLOv8, Modified Version, and Other YOLO Variants	56
4.11.1 Speed Comparison Between YOLOv8 and Improved YOLOv8	58
4.12 Conclusion	62
General Conclusion	63

List of Figures

1.1	Relationship between AI, ML and DL [5]	8
1.2	Object classification, localization and detection [36]	12
1.3	Multi-Object Tracking Pipeline: Detection to Trajectory Es- timation [17]	15
2.1	YOLO Object Detection Process [38].	20
2.2	YOLOv8 architecture [51].	23
3.1	C3TR Module Architecture [73].	36
3.2	Comparison between ADown downsampling and regular down- sampling [65].	38
3.3	Types of optimizations applied by TensorRT to enhance model performance [58].	40
3.4	Architecture of the Default YOLOv8 Model	42
3.5	Architecture of the Improved YOLOv8 Model	43
4.1	Football detection: Comparison Between YOLOv8 and Im- proved YOLOv8.	53
4.2	Safety Gear Detection: Comparison Between YOLOv8 and Improved YOLOv8.	54
4.3	Cow Detection Comparison Using Non-Augmented and Aug- mented Datasets	55
4.4	Sheep Detection Comparison Using Non-Augmented and Aug- mented Datasets	56
4.5	Comparison of Pituitary Tumor Detection Confidence Across YOLO Model Versions and Improved YOLOv8.	57
4.6	Comparison of Meningioma Detection Confidence Across YOLO Model Versions and Improved YOLOv8.	57
4.7	Comparison of Glioma Detection Confidence Across YOLO Model Versions and Improved YOLOv8.	57
4.8	Mask Detection Comparison Across YOLO Model Versions and Improved YOLOv8	58

4.9	Mask detection: Comparison Between PyTorch Model and TensorRT Model.	59
4.10	Glioma detection: Comparison Between PyTorch Model and TensorRT Model.	60
4.11	Comparison of OpenVINO and PyTorch models on real-world footage.	61

General Introduction

Object detection is everywhere now from self-driving cars and security cameras to medical imaging and smart factories. At the core of this tech is the challenge of quickly and accurately spotting objects in a scene. YOLO (You Only Look Once) has become one of the go-to solutions for this task because it's fast, simple, and surprisingly accurate.

That said, YOLO still struggles when things get messy like when objects are small, overlapping, or partially hidden. Real-world scenes are rarely clean, and this is exactly where models like YOLOv8 can start to fall short.

This thesis takes a closer look at how to make YOLOv8 smarter and more reliable in these complex situations. The goal isn't just to tweak some settings, but to actually improve its architecture and performance, especially in real-time scenarios. The focus is on making meaningful upgrades that keep the speed but boost the accuracy.

Here's how the thesis is organized:

- **Chapter 1 – Introduction:** Explains the background of object detection and AI, states the research problem, objectives, questions, and contributions.
- **Chapter 2 – Literature Review:** Reviews the evolution of YOLO models, especially YOLOv8, compares with other models, and highlights gaps in current research.
- **Chapter 3 – Methodology:** Explains the proposed improvements to YOLOv8, including architectural changes and design strategies aimed at boosting accuracy and speed.
- **Chapter 4 – Experimental Results and Analysis:** Presents the evaluation of the improved model through various tests, compares its performance with the original YOLOv8, and analyzes the results.

Chapter 1

Introduction

1.1 Introduction

Since our world relies more on smart technology, training machines to understand human vision is now a leading problem in computer science. What makes YOLO so popular among detection models is its ability to work fast and with surprising accuracy. In actual environments, there are many complicated things to notice. Even though they're messy, filled to capacity and can change quickly, YOLO still seems to have trouble. This chapter explains why bettering YOLO, especially in complex conditions, is both important and valuable. Also, it explains the motivation for the research, the main challenges faced and the main questions this thesis addresses.

1.2 Background and Motivation

1.2.1 Overview of Object Detection in Computer Vision

Detecting objects in an image or video is one of the main problems in computer vision and usually requires locating these objects exactly by marking them with bounding boxes. To cover many uses, this dual task is applied to automated surveillance, autonomous driving, robotics and augmented reality. Automatic perception of objects in the environment makes machines smarter when dealing with their world.

Before, object detection was done using features that experts designed, but this approach frequently had challenges with dealing with all types of real image data. Yet, when deep learning, particularly CNNs, appeared, it made it possible for this field to capture data in a better and scalable way. With these developments, models now work well across different data and places. [1, 2].

1.2.2 Importance of Real-Time Detection in Complex Scenes

In many practical scenarios, object detection must be performed not only accurately but also in real time. This is particularly critical in complex scenes such as urban environments, crowded public spaces, or cluttered indoor areas where numerous objects can appear simultaneously, often partially occluded, overlapping, or in varying scales and lighting conditions. For example, autonomous vehicles require instantaneous perception of the surroundings to detect pedestrians, vehicles, and traffic signs to make safe driving decisions

[3]. Similarly, security and surveillance systems rely on quick detection to monitor for suspicious activities without delay.

Achieving high detection speed while maintaining precision is challenging, especially under these demanding conditions. Real-time detection systems must strike a balance between computational efficiency and robustness to handle the inherent difficulties presented by clutter, occlusion, and environmental variations [21]. This motivates the development of models that can operate efficiently on edge devices and embedded systems, ensuring responsiveness without sacrificing accuracy.

1.2.3 Motivation for Choosing YOLO as the Baseline Model

YOLO which stands for “You Only Look Once,” is now considered a top and popular object detection framework. Unlike other detectors, YOLO solves the detection tasks as one unified regression problem by directly predicting the regions and classes in a single pass of the network. Thanks to this approach, the system detects intrusions very quickly, allowing for real-time use in applications. [21].

Each new version of YOLO has made the algorithm more accurate, faster and better designed. YOLOv8 includes new approaches to improve how detections are made. Nevertheless, there are still obstacles, especially with spotting small, dense or only partly visible things in complicated backgrounds. To face these problems, this thesis investigates ways to customize and enhance YOLOv8 to achieve both higher accuracy and efficiency during object detection. [6].

1.3 Problem Statement

1.3.1 Challenges Faced by YOLO in Complex Scenes

Although YOLO models have made great progress in the area of real-time object detection, they still struggle in complex, real situations. It can be difficult for a detection system to operate accurately in areas that are crowded, thick with vegetation or very messy. Different lighting, distracting sounds and varying look of objects add difficulties that YOLO must handle to deliver good results. That is why – due to those factors – YOLO can have trouble detecting or classifying objects effectively in the real world.

1.3.2 Limitations in Detecting Small, Overlapping, or Occluded Objects

YOLO often has a hard time detecting tiny objects that are crowded or huddled with nearby objects. Small objects have less data in an image and the model can't see many features to distinguish them. As a result of missing much image data, the system is less sure of its findings and detects fewer of those it should.

It is also difficult to identify objects in the scene when they are hidden by each other. If objects are close together and overlap, YOLO's output can detect only one object instead of two or even miss some objects. The challenge is very noticeable in crowds such as during heavy traffic or when many people gather, where good object detection is necessary.

All of these challenges together reveal where YOLO's performance can be improved in hard tasks and real-world environments.

1.4 Artificial Intelligence

Artificial Intelligence (AI) focuses on making computer systems capable of handling tasks that require human intelligence. Things they have to do include learning from experience, reasoning, solving challenges, perceiving and making decisions. AI makes it possible for machines to handle data, find patterns and make choices without humans and gets stronger by using Machine Learning (ML) and Deep Learning (DL) methods.

AI can be classified into two broad categories:

- **Narrow AI (Weak AI):** Designed for specific tasks, such as speech recognition (e.g., Siri, Google Assistant), image processing, or recommendation systems.
- **Strong AI (General AI):** A hypothetical concept in which machines would be given human-like cognitive abilities and be able to perform any intellectual task as well as a human.

Modern AI applications are widely used today in healthcare, finance, autonomous vehicles, robotics, and computer vision; thus, it is a fundamental technology of the contemporary digital era [39].

1.4.1 Domains of Artificial Intelligence

- **Natural Language Processing (NLP)**

NLP offers the capacity for machines to perceive, understand, and produce natural language. NLP combines computer science, AI, and linguistics to comprehend written and spoken words [42].

Important Techniques utilized in NLP:

- **Text Analysis:** Sentiment analysis, keyword extraction.
- **Speech to Text:** Converting speech to text (e.g., Google Voice, Siri).
- **Machine Translation:** Translating text (e.g., Google Translate).
- **Chatbots:** Smart customer service software. Uses include virtual assistants (Google Assistant), auto translators, and customer service robots.

Applications: Virtual personal assistants (Google Assistant), computer-aided translators, customer support bots.

- **Computer Vision**

Computer Vision allows AI machines to interpret and analyze visual information from the external world. Computer Vision enables object identification, facial recognition, motion detection, and image and video analysis [43].

Key Computer Vision Methods:

- **Object Detection:** Detection and classification of objects in an image.
- **Facial Recognition:** Identification and verification of individuals based on facial features.
- **Image Segmentation:** Separation of objects in an image for examination.

Applications: Self-driving cars (Tesla), security surveillance, medical imaging (AI diagnosis).

Robotics and Automation

Robotics is the use of AI together with mechanical and electrical engineering to create intelligent machines that can perform tasks independently. AI-based robots can perceive their environment, make decisions, and execute actions effectively [44].

Key Technologies in Robotics:

- **Self-Navigation:** Used in autonomous vehicles and drones.
- **Humanoid Robots:** Robots that imitate human movements (e.g., Sophia the Robot).
- **Factory Automation:** AI-based robots in manufacturing (e.g., robotic arms in factories).

Applications: Factory robots, autonomous drones, medical robotics (surgical robots).

Expert Systems

Expert Systems are AI programs intended to mimic human expertise in certain domains. They utilize knowledge bases and inference engines to make intelligent decisions [40].

Components of Expert Systems:

- **Knowledge Base:** Holds domain rules and facts.
- **Inference Engine:** Uses reasoning to examine input and produce conclusions.
- **User Interface:** Facilitates interaction between users and the system.

Applications: Medical diagnosis (AI-powered healthcare advisers), financial forecasting, legal decision support.

Fuzzy Logic

Fuzzy Logic is a computational approach that deals with reasoning under uncertainty. Unlike traditional binary logic (true/false), fuzzy logic allows for degrees of truth, making AI systems more adaptable to real-world situations [45].

Key Aspects of Fuzzy Logic:

- Handles imprecise and ambiguous data.
- Used in control systems where precise values are hard to determine.

Applications: AI-powered HVAC systems, automotive control systems, weather forecasting.

Reinforcement Learning (RL)

Reinforcement Learning is a domain of AI where agents learn to make optimal decisions by interacting with an environment and receiving rewards or penalties based on their actions [46].

Key Concepts in RL:

- **Agent:** The AI system making decisions.
- **Environment:** The world in which the agent operates.
- **Reward Function:** Guides the agent toward the best actions.

Applications: Robotics, gaming AI (AlphaGo), financial trading bots.

1.4.2 Specialized Areas of Artificial Intelligence

Artificial Intelligence (AI) covers the general area of creating smart machines, while Machine Learning (ML) is one branch that trains them using data. Deep Learning (DL) is an advanced ML approach that uses deep neural networks.

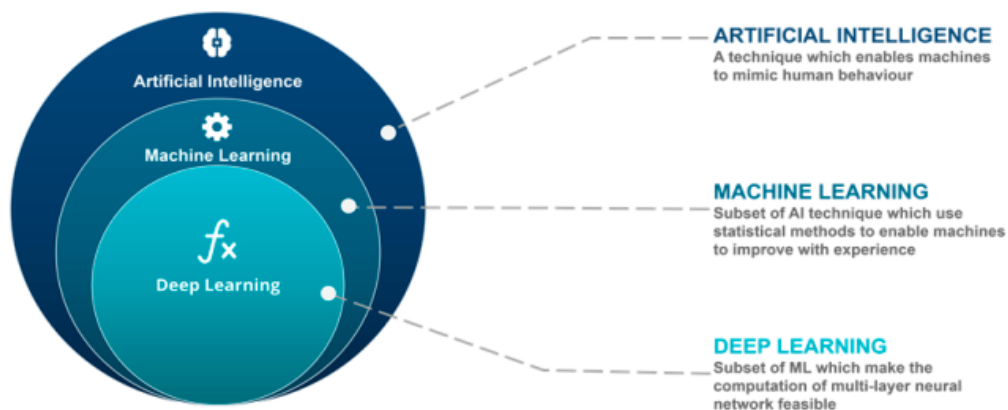


Figure 1.1: Relationship between AI, ML and DL [5]

- **Machine Learning (ML)**

With Machine Learning (ML), systems are able to learn and improve by using data instead of being programmed by hand. Instead of being told what to do by rules, ML algorithms look for patterns in the data and use them to make decisions or predictions.

Many industries, for example, healthcare, finance and autonomous systems, use ML because it can quickly process a lot of data [47, 48].

Main Features of Machine Learning:

- Capability to learn and enhance over time according to new data.
- Automates decision-making without human intervention.
- Draws conclusions and patterns from complex datasets.

Types of Machine Learning:

- **Supervised Learning:** The model is trained using labeled data, which means that it learns from input-output pairs where the correct answer is provided.
Examples: Spam email detection, fraud detection, speech recognition.
- **Unsupervised Learning:** The model is trained using unlabeled data, detecting hidden patterns or structures within the data without pre-established categories.
Examples: Customer segmentation, anomaly detection, recommender systems.
- **Reinforcement Learning:** It learns through interacting with an environment and getting rewarded or penalized depending on its actions, making its decisions optimal over time.
Examples: Game artificial intelligence (AlphaGo), automation robots, driverless cars.

Applications of Machine Learning:

- **Healthcare:** AI-based diagnosis, analysis of medical images, and individualized treatment plans.
- **Finance:** Credit scoring, algorithmic trading, fraud detection.
- **E-commerce:** Recommender systems (Amazon, Netflix), dynamic pricing strategies.
- **Autonomous Cars:** Autonomous car technology that uses ML in object recognition and route planning.

Deep Learning (DL)

In Deep Learning (DL), machines imitate the human brain by using neural networks to handle and understand a lot of challenging data. With pattern recognition applicable areas, DL models can automatically form a structure of different-level representations. While manual feature engineering is needed with traditional ML, deep learning can discover useful features by itself and thus works well for processing images, speech and text[49, 50].

Main Features of Deep Learning:

- Employs multi-layered neural networks (deep neural networks) to depict complex patterns.
- Requires huge datasets and significant computational power for effective training.
- Can automatically extract and enhance features from raw input data.

Types of Deep Learning Architectures:

- **Convolutional Neural Networks (CNNs):** Designed for image and video analysis, CNNs are used in facial recognition, medical imaging, and object detection.
- **Recurrent Neural Networks (RNNs):** Employed for sequential data processing, e.g., speech recognition and language modeling. Variants like Long Short-Term Memory (LSTM) networks improve memory retention.
- **Transformer Models:** Powerful models employed in natural language processing (NLP) tools, like GPT (used in ChatGPT) and BERT (applied in Google search optimization).

Applications of Deep Learning:

- **Computer Vision:** Face identification, autonomous vehicle perception, and medical diagnosis (X-ray reading using AI).
- **Speech and Language Processing:** Virtual assistants (Siri, Alexa), language translation (Google Translate), chatbots.
- **Generative AI:** AI-generated deepfakes, AI-created art, and text generation models.
- **Healthcare:** Disease prediction using AI, drug discovery, robotic-assisted surgery.

1.5 Object Detection and Tracking

1.5.1 Object Detection

A part of computer vision called object detection helps to discover and mark the locations of objects within an image or one frame of a video. Whereas image classification only tells if an object is present, object detection also outlines each detected object with a boundary. Sensory perception is key for handling assistance and for robots, monitoring, car driving and scanning devices [52, 54]. Object detection consists of two main tasks:

- **Image Classification:** Image classification involves labeling or classifying an entire image based on the objects present within it. Deep learning models, such as Convolutional Neural Networks (CNNs), process the image and output a class label. However, image classification does not provide any spatial information about the location of the object in the image, making it insufficient for applications that require precise localization [55].
- **Image Localization:** Image localization extends image classification by identifying the position of an object within an image. It typically outputs bounding box coordinates around the detected object, providing better spatial understanding. Traditional image localization techniques are often limited to detecting a single object per image and may not perform well when multiple objects are present [56].

Beyond these tasks, modern object detection methods combine both classification and localization to detect multiple objects in an image simultaneously. Recent deep learning-based approaches have significantly improved the accuracy and efficiency of object detection, enabling real-time performance across diverse applications.

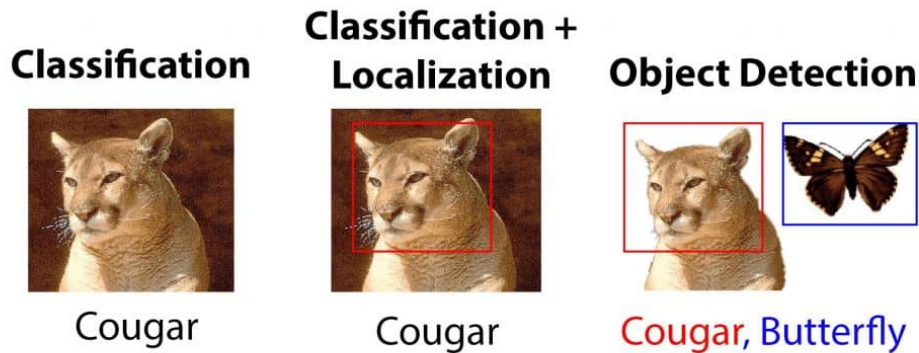


Figure 1.2: Object classification, localization and detection [36]

Popular Object Detection Models

Several deep learning-based object detection models have gained popularity, including:

- **YOLO (You Only Look Once):** YOLO is a real-time object detection model that divides the image into a grid and simultaneously predicts class probabilities and bounding boxes. It is very fast and efficient, making it suitable for real-time applications [57].
- **R-CNN (Region-based Convolutional Neural Networks):** R-CNN is a family of object detection models based on region proposal methods that identify possible object locations before classifying them. Its variants include Fast R-CNN, Faster R-CNN, and Mask R-CNN, all improving detection speed and accuracy [59].
- **SSD (Single Shot MultiBox Detector):** SSD is another object detection model that, like YOLO, predicts bounding boxes and class probabilities in a single network pass. It offers a good balance between speed and accuracy [60].

Feature	YOLO	RCNN	SSD
Detection Speed	High (Real-time)	Low (Slow)	Medium
Accuracy	Moderate	High	High
Architecture	Single Network	Multiple Stages	Single Network
Region Proposal	No	Yes	No
Complexity	Low	High	Medium
Applications	Real-time Processing	High-precision Tasks	Balanced Performance
Limitations	Lower Accuracy for Small Objects	Computationally Expensive	Less Accurate than RCNN

Table 1.1: Comparison of YOLO, RCNN, and SSD for Object Detection [21, 22, 23]

1.5.2 Object Tracking

Object tracking is a major issue in computer vision because it requires continuously following objects as they change position frame by frame in a video. Object detection looks at just one frame, but object tracking connects previous and current detections to see if they are the same object. This makes it possible to use data immediately for video surveillance, autonomous driving, controlling systems with people and understanding sports games[61, 71].

Object tracking generally involves two main steps:

- **Detection and Initialization:** In the first frame, an object detection model identifies objects and initializes tracking by assigning unique identifiers to each detected object.
- **Tracking and Update:** After initialization, tracking algorithms update each object’s location in subsequent frames, maintaining its identity despite challenges such as occlusion, scale changes, and motion blur.

Tracking approaches differ in their robustness to occlusion, appearance changes, and object interactions.

Types of Object Tracking

- **Single Object Tracking (SOT):** Tracks a single object throughout a video sequence. It tends to be highly accurate but is not suitable for scenarios involving multiple objects [72].

- **Multiple Object Tracking (MOT):** Simultaneously tracks multiple objects, maintaining their identities over time. Common applications include traffic monitoring, pedestrian detection, and sports analytics [74].
- **Correlation Filter-Based Tracking:** Uses correlation filters to track objects based on appearance features. While efficient, it can struggle with significant appearance changes [75].
- **Deep Learning-Based Tracking:** Employs neural networks for robust tracking, often combining object detection with Recurrent Neural Networks (RNNs) or attention mechanisms to maintain consistency across frames [76].
- **Kalman Filtering and Particle Filtering:** Traditional probabilistic methods for estimating object motion. Kalman filters assume linear motion with Gaussian noise, whereas particle filters handle nonlinear dynamics and complex interactions [77, 78].
- **Optical Flow-Based Tracking:** Utilizes pixel motion estimation between consecutive frames to determine object trajectories. Optical flow is effective in dynamic scenes but computationally intensive [79].
- **Feature-Based Tracking:** Tracks objects by detecting distinctive feature points such as SIFT (Scale-Invariant Feature Transform) or SURF (Speeded-Up Robust Features). This method is resilient to scale and rotation changes but sensitive to occlusions [80].

Ongoing advances in deep learning and optimization methods continue to enhance object tracking performance, cementing its role as a core technology in modern computer vision systems.

1.6 Research Objectives

The primary goal of this thesis is to enhance the accuracy of the YOLO (You Only Look Once) object detection model, especially in complex and challenging scenes where traditional detection methods often struggle. YOLO has become a widely used baseline for real-time detection tasks due to its balance of speed and accuracy [21].

To achieve this goal, several specific objectives are pursued:

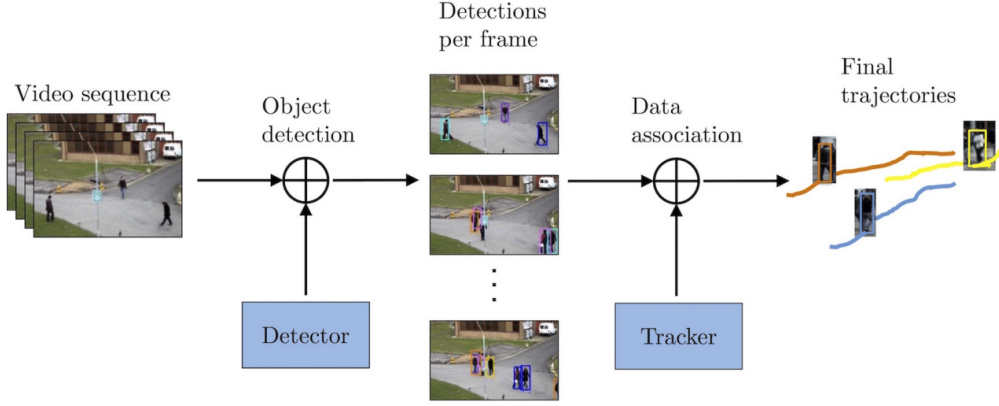


Figure 1.3: Multi-Object Tracking Pipeline: Detection to Trajectory Estimation [17]

- **Architecture Modifications:** Proposing and integrating novel network components, such as attention mechanisms, which have proven effective in enhancing feature representation [9], and optimizing backbone structures.
- **Performance Optimization:** Balancing detection accuracy with real-time inference speed to maintain YOLO’s practicality for applications like autonomous driving and surveillance [8].

These objectives form the foundation of this research, aiming to push YOLOv8’s capabilities to better handle complex environments and challenging detection scenarios.

1.7 Research Questions and Hypotheses

The study is guided by several key research questions:

- Can the integration of attention mechanisms into YOLOv8’s architecture significantly improve detection accuracy in cluttered or occluded scenes?
- What architectural modifications maximize performance without sacrificing inference speed?

Based on these, the hypotheses are:

- Incorporating attention modules will enable the model to focus more effectively on relevant features, resulting in measurable accuracy improvements [9].
- We think that by using tools like TensorRT and OpenVINO to optimize our models, it's possible to get better performance without slowing things down—in fact, they might even run faster.

1.8 Contributions of the Study

This thesis offers the following key contributions:

- Development and evaluation of novel attention-based modules customized for YOLOv8 to enhance feature representation [9].
- Extensive performance evaluation showing enhanced accuracy and efficiency compared to the original YOLOv8 baseline [21].

1.9 Conclusion

The significance of object detection was explained, strengths and weaknesses of YOLO were presented and the necessity to enhance it was made clear in this chapter. It started by introducing the research objectives and major questions which leads to the in-depth technical and experimental sections in the next chapters. Following chapters discuss solutions to make YOLOv8 operate well in tough and real world settings.

Chapter 2

Literature Review

2.1 Introduction

Before diving into improving YOLO, it's important to understand where it all started and how it evolved. This chapter takes a close look at YOLO's journey from its first version to the latest YOLOv8, explaining how it works under the hood and what makes it stand out. We also explore how well it performs, especially when speed and accuracy are both on the line. But no model is perfect—especially in tricky, crowded scenes—so we review the main challenges researchers have faced. Finally, we look at the different ways experts have tried to push YOLO's accuracy further, from architectural tweaks to smart tricks like attention mechanisms and ensemble learning. This sets the stage for identifying what's been done and where we can still make a real difference.

2.2 Overview of YOLO Architecture

2.2.1 YOLO Evolution: YOLOv1 to YOLOv8

YOLOv1: Breakthrough in Real-Time Object Detection

Initially proposed by Joseph Redmon et al. in 2016, YOLOv1 was a single-shot detector that framed object detection as a regression problem, predicting bounding boxes and class probabilities directly from an image in a single forward pass. This enabled real-time performance but sacrificed accuracy in small object detection and localization precision[81].

YOLOv2 and YOLOv3: Enhancing Accuracy and Speed

YOLOv2 (2017) improved upon YOLOv1 by introducing batch normalization, anchor boxes, and a resolution-enhancing backbone (Darknet-19). These improvements led to increased accuracy without sacrificing speed[82].

YOLOv3 (2018) introduced a multi-scale detection strategy using Feature Pyramid Networks (FPN) and adopted Darknet-53, a more efficient and powerful network. YOLOv3 provided better accuracy, particularly for small objects[82].

YOLOv4 and YOLOv5: Performance Optimization

YOLOv4 (2020), introduced by Alexey Bochkovskiy, integrated CSPDarknet-53, Mish activation, and spatial pyramid pooling (SPP), significantly boosting both speed and precision[82].

YOLOv5 (2020), developed by Ultralytics, prioritized ease of deployment by introducing a PyTorch implementation, scalable models, and fast training pipelines, quickly becoming a developer favorite[82].

YOLOv6 and YOLOv7: High-end Features and Refinements

YOLOv6 targeted industrial applications by introducing anchor-free detection and an upgraded backbone to optimize performance on mobile and edge devices[82].

YOLOv7 focused on scalable model efficiency, achieving state-of-the-art accuracy while guaranteeing real-time execution, surpassing many previous versions in both speed and accuracy[82].

YOLOv8: A Major Evolution in the YOLO Family

YOLOv8 is considered one of the most reliable models in the YOLO family due to its consistent performance, featuring[82]:

- Anchor-free detection for better adaptability to various object sizes.
- Improved Feature Pyramid Networks (FPN) for enhanced multi-scale detection.
- Optimized training strategies that improve accuracy and generalization.
- Multiple model variants (YOLOv8n, YOLOv8s, YOLOv8m, etc.) to balance speed and precision for different applications.

2.2.2 How YOLO Works

YOLO (You Only Look Once) is a real-time object detection algorithm that frames detection as a one regression problem, mapping input images directly to class probabilities and bounding boxes in a single pass[91, 95]. In contrast to region-based methods, YOLO detects an image in a single pass, making it highly efficient[92, 93].

Grid-based Object Detection

YOLO divides an input image into an $S \times S$ grid. Each grid cell is responsible for object detection whose center is inside the cell. Each cell predicts:

- A fixed number of bounding boxes.
- Confidence scores for each bounding box.
- Class probabilities for each detected object.

This approach allows YOLO to detect many objects in an image efficiently[94].

Bounding Box Prediction

Each bounding box prediction has:

- **Coordinates** (x, y, w, h) : The center (x, y) and width/height (w, h) of the box.
- **Confidence Score**: Indicates the likelihood that an object is present and the accuracy of the bounding box.
- **Class Probabilities**: The probability distribution over all possible object classes.

The final predictions are refined using Non-Maximum Suppression (NMS) to remove duplicate detections[94].

Single Forward Pass for Detection

YOLO processes the entire image in a single forward pass through a convolutional neural network (CNN), extracting features and predicting objects simultaneously. This eliminates the need for multiple region proposals and classification steps, which makes YOLO one of the fastest object detection methods[92, 91].

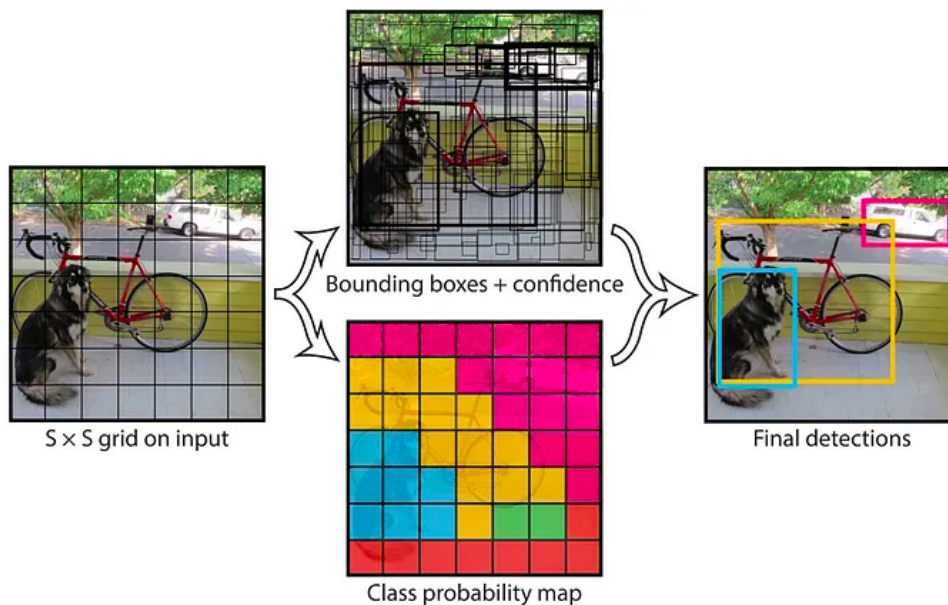


Figure 2.1: YOLO Object Detection Process [38].

Advantages of YOLO

- **Fast:** YOLO is able to run in real time, and hence it is suitably conditioned to deal with applications like autonomous driving and monitoring.
- **Global Context:** Since the entire image is processed simultaneously, YOLO gains contextual understanding of things.
- **High Accuracy:** Despite its speed, YOLO achieves high performance through optimizing feature extraction and bounding box estimation.

This single-stage detection model has made YOLO the most widely used object detection model employed on computer vision tasks.

2.2.3 YOLOv8 Architecture

Backbone

The backbone in YOLOv8 is based on an optimized version of CSPDarknet, designed to extract meaningful features while maintaining efficiency[83].

CSPDarknet Architecture

The CSPDarknet backbone includes several key elements:

- **Convolutional Layers:** Standard 3×3 and 1×1 convolutional filters are used to extract spatial features.
- **CSP (Cross Stage Partial) Blocks:** These blocks divide the feature map into two paths: one undergoing dense computations and the other bypassing it to reduce computational overhead.
- **Residual Connections:** Inspired by ResNet, CSP blocks maintain gradient flow and prevent vanishing gradients.
- **SiLU Activation:** The Swish activation function is used:

$$SiLU(x) = x \cdot \sigma(x), \quad \sigma(x) = \frac{1}{1 + e^{-x}} \quad (2.1)$$

- **SPPF (Spatial Pyramid Pooling - Fast):** Instead of using multiple kernel sizes, YOLOv8 applies a single 5×5 max pooling operation three times sequentially to capture multi-scale features efficiently.

Neck

The neck of YOLOv8 enhances feature extraction by integrating multiple scales and improving feature fusion[84]. It consists of:

Feature Pyramid Network (FPN)

FPN is used to propagate low-level features to higher levels, enabling small object detection.

$$F_{i+1} = \text{Upsample}(F_i) + \text{Conv}(F_{i-1}) \quad (2.2)$$

where F_i is the feature map at level i .

Path Aggregation Network (PAN)

PAN further refines the feature maps by adding bottom-up and top-down connections, strengthening localization features.

$$F_{out} = \text{Concat}(F_{PAN}, F_{FPN}) \quad (2.3)$$

This enables efficient feature propagation across different scales.

Head

The head in YOLOv8 is responsible for making predictions. Unlike previous YOLO versions, YOLOv8 employs an **anchor-free approach**, improving speed and accuracy[85].

Bounding Box Regression

Bounding boxes are predicted using a direct regression method:

$$(x, y, w, h) = \text{Sigmoid}(F) \times S \quad (2.4)$$

where S is the stride of the feature map.

Classification

Each detected object is assigned a probability score p_i for class i , using the sigmoid activation function:

$$p_i = \frac{1}{1 + e^{-z_i}} \quad (2.5)$$

where z_i is the raw classification score.

2.3 Performance Analysis

2.3.1 Accuracy and Speed Trade-offs

YOLOv8 seeks to find a balance between detection accuracy and computational efficiency and hence is appropriate for real-time application in applications such as autonomous driving, video surveillance, and robotics. The accuracy-speed trade-off is based on a number of factors

- **Model Complexity:** YOLOv8 provides a range of model variants (e.g., YOLOv8n, YOLOv8s, YOLOv8m, YOLOv8l, YOLOv8x) to fit various applications. Large models have more parameters as well as deeper networks, which results in higher accuracy but also increases computational complexity.

Model Name	Params (M)	Accuracy (mAP 0.5)	CPU Time (ms)	GPU Time (ms)
YOLOv8n	3.2	50.2	38	5.8
YOLOv8s	11.2	61.0	87	6.1
YOLOv8m	25.9	67.5	195	7.9
YOLOv8l	43.7	70.2	400	9.5
YOLOv8x	68.2	71.7	720	11.4

Table 2.1: YOLOv8 Model Performance Metrics [53]

- **Anchor-free Detection:** Unlike previous models which relied on a set of pre-defined anchor boxes, YOLOv8 is anchor-free. This simplifies training, reduces memory usage, and detection becomes more flexible, especially for small and irregularly shaped objects.
- **Optimization Techniques:** YOLOv8 employs improved training techniques such as mosaic augmentation, label smoothing, and improved loss functions (e.g., CIoU Loss) to enhance accuracy as well as robustness without sacrificing high-speed inference.
- **Hardware Optimization:** The architecture is optimized for GPUs and edge devices, enabling efficient deployment on a range of hardware, from embedded AI chips like NVIDIA Jetson to edge TPUs.

2.3.2 Comparison with YOLOv7 and Other Object Detection Models

YOLOv8 offers significant improvements over YOLOv7 and other object detection models by means of efficiency, accuracy, and real-time operation. The comparison is as shown in the table below:

- **YOLOv8 vs. YOLOv7:** YOLOv8 enhances feature extraction and detection by enhancing the backbone, neck, and head modules. It employs a more efficient feature pyramid network (FPN) and an anchor-free detection system, which gives improved accuracy and reduced computational overhead.
- **YOLOv8 vs. Two-stage Models:** There exists a slower two-stage object detector, such as Faster R-CNN, which relies on a region proposal network (RPN) followed by classification, thus slower than YOLO's one-stage approach. YOLOv8 can process images in a single pass, achieving real-time performance.
- **YOLOv8 vs. EfficientDet:** While EfficientDet is good accuracy with fewer parameters, and therefore slower than YOLOv8 since it employs more complicated feature fusion strategies. YOLOv8 provides better accuracy vs speed trade-off.

Model	mAP@50	FPS	Anchor-free	Parameters
YOLOv7	56.8%	161 FPS (~6 ms)	No	37M
YOLOv8	59.2%	177 FPS (~5.6 ms)	Yes	36M
Faster R-CNN	55.0%	~7 FPS (~140 ms)	No	42M
EfficientDet	57.5%	~40 FPS (~25 ms)	No	20M

Table 2.2: Comparison of YOLOv8 with YOLOv7 and other object detection models [24, 25, 26, 27].

2.3.3 Evaluation Metrics (mAP, FPS, Latency)

The performance of YOLOv8 is measured using several key metrics:

Precision and Recall

Precision and recall are fundamental metrics for evaluating object detection performance:

- **Precision:** Measures the proportion of correctly predicted objects among all detected objects. It is defined as:

$$Precision = \frac{TP}{TP + FP} \quad (2.7)$$

where:

- TP (True Positives) are correctly detected objects.
 - FP (False Positives) are incorrect detections (e.g., detecting an object that does not exist).
- **Recall:** Measures how many actual objects were correctly detected. It is given by:

$$Recall = \frac{TP}{TP + FN} \quad (2.8)$$

where:

- FN (False Negatives) are objects that were present but not detected.

A higher precision means fewer false detections, while a higher recall means the model is detecting more of the actual objects.

Intersection over Union (IoU)

Intersection over Union (IoU) is used to evaluate how well a predicted bounding box aligns with the ground truth bounding box:

$$IoU = \frac{AreaofOverlap}{AreaofUnion} \quad (2.9)$$

A higher IoU indicates better localization of objects.

Mean Average Precision (mAP)

The primary metric used for object detection evaluation is the Mean Average Precision (mAP). It is calculated by integrating precision over recall:

$$AP = \int_0^1 P(R) dR \quad (2.10)$$

where $P(R)$ represents the precision-recall curve. The mean Average Precision (mAP) is computed as the average of the AP values across all object classes:

$$mAP = \frac{1}{N} \sum_{i=1}^N AP_i \quad (2.11)$$

where N is the number of classes.

Frames Per Second (FPS)

Frames Per Second (FPS) measures the speed of the model:

$$FPS = \frac{TotalProcessedFrames}{TotalProcessingTime} \quad (2.12)$$

A higher FPS value indicates faster real-time processing.

Latency

Latency refers to the time taken for a single image to be processed and produce an output. It is typically measured in milliseconds (ms):

$$Latency = \frac{1}{FPS} \quad (2.13)$$

Lower latency is crucial for real-time applications such as autonomous vehicles and surveillance.

These metrics collectively determine the performance of YOLOv8, ensuring a balance between accuracy and speed for real-world applications.

2.4 Challenges in Complex Scene Detection

Object detection in complex scenes remains one of the toughest challenges in computer vision. Real-world environments are rarely simple or controlled. Various factors such as lighting variation, background clutter, and occlusion make accurate detection very difficult for even the most advanced algorithms.

2.4.1 Core Obstacles in Complex Scene Detection

Lighting Variation

It is hard to predict the amount of light in real-world scenes. The lighting conditions, from bright sun, shadows, mirrors or inside, can harshly affect how objects look. The same car exposed to full sunlight looks much brighter than it does when in the shade or at dusk. Lighting problems can affect what the model identifies and often lead to either missing what it is supposed to see or seeing something that is not there. [15]. Robust models must therefore be able to adapt to a wide range of lighting environments to maintain accuracy.

Background Clutter

Many objects or textures can overlap or surround the important objects in an image which is known as background clutter. A lot of trees, signs, buildings and people in an urban or natural scene can make it tough for an algorithm to detect the right object. That's often what makes it hard to identify the object from among other signals[16]. Overcoming this clutter requires models that can learn strong discriminative features and contextual cues.

Occlusion

When an object is covered by something else, part of it becomes hard to see which is called occlusion. Obstacles such as vehicles or street furniture can cover pedestrians and vegetation may hide animals in the scene. Because many techniques for finding objects depend on seeing the entire shape or a certain feature, obstruction reduces what the model can find, leading to mistakes [15].

2.4.2 Real-world Dataset Examples

Datasets like MS COCO [16] and Cityscapes show a realistic combination of all these difficulties, as they include pictures with lots of objects in crowded areas under different lighting. Having datasets like these is necessary for training and benchmarking models because they match the real-life situations that the detection device will face. There is still a big difference between how AI performs in tests and how it works in real applications because of these factors.

Fixing these problems helps detection systems last longer and be more accurate which is especially important for things like self-driving cars, surveillance cameras and robots.

2.5 Related Work on Improving YOLO Accuracy

YOLO has continuously been progressed to be able to spot objects in challenging scenes. Many approaches have been explored to make the model better at identifying but still work in a timely manner.

2.5.1 Model Architecture Enhancements

Architectural improvements focus on enabling better feature extraction and multi-scale representation. The Cross Stage Partial Network (CSPNet) [19] divides the feature map into two parts and merges them through a cross-stage hierarchy, improving gradient flow and reducing computational bottlenecks. Path Aggregation Networks (PANet) [20] enhance feature fusion from different layers, allowing the model to capture both high-level semantics and fine details critical for small or occluded object detection. These improvements help YOLO variants perform more accurately on complex scenes without sacrificing speed.

2.5.2 Integration of Attention Mechanisms

Attention modules, such as Squeeze-and-Excitation (SE) blocks [28], Convolutional Block Attention Modules (CBAM) [29], and Transformer blocks [30], have been integrated into YOLO to allow the network to focus on important features dynamically. These mechanisms weigh spatial and channel-wise features differently depending on context, enabling the model to highlight relevant object regions and suppress background noise. Attention modules are

particularly effective in cluttered scenes where distinguishing objects from complex backgrounds is challenging.

2.5.3 Ensemble Learning and Post-processing Methods

Combining outputs from multiple models or using advanced post-processing techniques, such as improved Non-Maximum Suppression (NMS), helps reduce false positives and improve detection precision [31]. Ensemble learning leverages the strengths of different architectures or training configurations, while refined post-processing eliminates overlapping detections more effectively, enhancing final prediction quality.

Collectively, these advances represent a significant step forward for YOLO’s performance in real-world conditions, balancing the trade-off between accuracy and speed crucial for practical applications.

2.6 State of the Art

The field of real-time object detection has undergone rapid advancements over the last decade, largely driven by the YOLO (You Only Look Once) family of models.

YOLOv1 (2016) – Joseph Redmon et al.

The original YOLO, introduced by Joseph Redmon and his team [10], This work made object detection easier by suggesting a single-stage framework for all. YOLO uses a regression approach instead of region proposal networks or sliding windows which allows for detecting objects faster in real time. This method simplified the process and resulted in fast inference along with good accuracy.

YOLOv3 (2018) – Joseph Redmon and Ali Farhadi

Building on YOLOv1, Redmon and Farhadi [11] released YOLOv3 and made major changes by adding multi-scale predictions, updating the backbone (to Darknet-53) and introducing logistic regression for objectness prediction. Greatly strengthening YOLOv3 made it both more accurate with little objects and quick enough to be used on everyday video tasks.

YOLOv4 (2020) – Alexey Bochkovskiy, Chien-Yao Wang, and Hong-Yuan Mark Liao

YOLOv4 [12] focused on optimizing the trade-off between speed and accuracy. It incorporated “bag-of-freebies” techniques like data augmentation, new activation functions, and optimized training strategies. YOLOv4 deliv-

ered state-of-the-art results with efficient computational requirements, making it widely adopted in the object detection community.

YOLOv7 (2022) – Chien-Yao Wang et al.

YOLOv7 [13] introduced novel training tricks and architectural improvements that further pushed the performance boundaries for real-time detectors. The model improved accuracy without sacrificing inference speed, including enhanced model scaling and training strategies.

YOLOv8 (2023) – Ultralytics Team

YOLOv8 [14] offers a modular architecture and streamlined training pipeline, simplifying customization and deployment. It emphasizes robustness and adaptability to a variety of detection tasks, contributing to its strong performance across different domains.

Despite these advances, challenges remain in detecting small, occluded, or densely packed objects. This thesis aims to improve YOLOv8’s architecture and training methods to enhance detection accuracy and robustness in such complex scenarios.

2.7 Summary and Research Gaps

Current research shows that YOLO can now detect objects efficiently in hard-to-analyze situations. Millions of images used for training and advancements in Convolutional Squeeze-and-Excitation networks (CSPNet) and PANet, along with attention mechanisms and ensemble methods, have all led to increased performance. But, some significant gaps are still there.

A lot of existing methods have trouble discovering small, overlapping or very blocked objects often seen in places crowded with objects. Also, many of the proposed changes require more computational power which may make it harder for YOLO to work in real time on edge devices.

The study strives to fill these gaps by exploring changes to the architecture and special care modules intended to raise the accuracy of detection in tough environments without decreasing speed. As a result of solving these issues, YOLOv8 will be more solid, effective and useful for being implemented in real-life tasks.

- **Difficulty with Small Object Detection:** The anchor-free method enhances detection flexibility but may have difficulty detecting small objects in crowded scenes. This can be a problem in use cases such as aerial surveillance and fine-grained object recognition.
- **High Computational Demands:** Larger YOLOv8 models require a lot of GPU resources, and deployment becomes difficult on low-power

devices. Although the smaller versions (e.g., YOLOv8n) compromise computational load, they compromise accuracy.

- **Challenging Environment Performance Degradation:** YOLOv8 may degrade lower accuracy under low-light, bad weather, or high-density background conditions. This may affect applications like night surveillance and outdoor robotics.
- **Accuracy-Speed Trade-off:** YOLOv8, while low-latency, may not always beat transformer-based models in high-accuracy applications. It is tailored for real-time processing but may not achieve maximum possible accuracy in every situation.
- **Bias and Ethical Concerns:** Like other deep learning models, YOLOv8 can inherit the training data biases. This is an ethical concern in facial recognition and law enforcement, where biased predictions could lead to discrimination.

2.8 Conclusion

The chapter looked at how the YOLO architecture has developed and how it works, mainly discussing YOLOv8. Throughout, it looked at how the different models performed, how accurate they were and how easy they were to use in different settings. Work was also done to examine existing investigations into better detection when circumstances are difficult. The field continues to face some important challenges as it tries to use YOLOv8 in the real world. Using what we learned, we will present our suggested plan in the next chapter.

Chapter 3

Methodology

3.1 Introduction

This chapter goes over how the enhanced YOLOv8 model was designed and put into action. The explanation starts by stating that YOLOv8 was chosen for its ability to adapt. After that, the chapter explains the improvements planned for the network's architecture, such as the addition of C3TR, AD-own, and dual SPPF modules to raise accuracy while maintaining speed. It further lists the strategies (TensorRT, OpenVINO) chosen to support real-time performance and assesses how these were verified during testing.

3.2 Justification for Using YOLOv8

The main reason for choosing YOLOv8 is that it offers balance among performance, modularity and flexibility in putting it into practical use. YOLO first made it possible to detect objects in real time and now YOLOv8 goes a step further by being well-suited for dealing with complicated and varied datasets.

Not using anchors in the design makes training easier and ensures the model detects different-sized and shape objects more accurately. This feature helps a lot when you are working with datasets that include unusual objects that are unique to a field such as medical imagery and agricultural data. Because YOLOv8 separates the head into classification and regression branches, it is able to localize and predict labels with better accuracy.

The modular system in YOLOv8, with modules including the C2f, is another key reason to pick this architecture. With these components, it is easier to extract important data and the model can perform well without using too much computing power. YOLOv8 includes built-in support for training, validating and converting models to many formats (like ONNX, TorchScript and TensorRT) which makes it suitable for use and deployment in different areas.

Being able to deploy it easily using lightweight tools and making it fit for various purposes increased its chances of being used here. These benefits fit well with the goal of this study which is to examine better object detection performance across different areas by using YOLO-based models.

3.3 Proposed Improvements to YOLO

3.3.1 Architectural Modifications for Enhanced Accuracy

To increase the accuracy of YOLOv8, we suggest new architectures for the backbone, the neck and the detection head. With these updates, the model can recognize close details and also plan the scan of remote details in the image, helping to find objects in different scenes more effectively.

- **The C3TR Block: A Hybrid CNN-Transformer Module**

A central innovation in our approach is the introduction of the **C3TR** (Cross-Stage Partial Network with Transformer) block. This hybrid module integrates Transformer layers into the conventional C3 block originally used in YOLOv8. The motivation behind this design is to leverage the complementary strengths of Convolutional Neural Networks (CNNs) and Transformers. CNNs excel at extracting local features such as edges, textures, and fine details by focusing on small, localized image patches. However, they struggle to model relationships between distant parts of an image. In contrast, Transformers provide a mechanism for capturing long-range dependencies and global context through self-attention mechanisms.

The C3TR block achieves this by splitting input features into two parallel branches:

- **Regular Vision Branch (Convolutions):** Focuses on local feature extraction by processing small image regions. This branch detects fine-grained textures and edges essential for object localization and classification.
- **Global Vision Branch (Transformer):** Captures global relationships by attending to all regions of the image simultaneously. This enables the model to connect spatially distant but semantically related features, which is particularly beneficial in complex scenes with occlusions or overlapping objects.

Together, these two branches act as a complementary team: the convolutional path serves as a “Detail Detective,” while the Transformer path functions as a “Big-Picture Detective.” This synergy significantly enhances the model’s ability to recognize challenging objects and improves overall detection robustness.

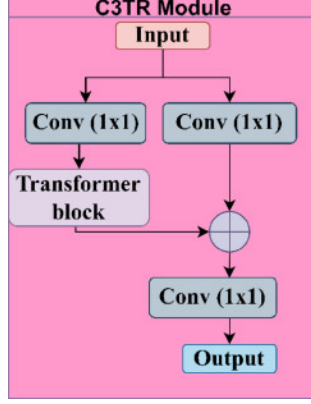


Figure 3.1: C3TR Module Architecture [73].

- **Backbone Enhancements**

In addition to integrating the C3TR block, the backbone architecture is deepened to extract richer hierarchical features. The number of C2f blocks in the early layers is expanded from 3–5 to 5–8 in the middle layers, providing a more comprehensive representation of complex patterns. Residual connections within these blocks ensure stable gradient flow during training, preventing issues such as vanishing gradients which can degrade performance in deeper networks. This adjustment is motivated by the philosophy of “quality over quantity” which means instead of increasing the number of feature maps through the addition of extra convolutional layers or C2f modules, our approach prioritizes improving the representational quality of the existing feature maps. This strategy aims to enhance feature extraction efficiency without significantly increasing computational complexity.

Crucially, the standard C2f block at layer 6 (P4/16 scale) is replaced by the transformer-based C3TR block. This targeted substitution infuses global contextual awareness directly into the backbone, allowing the network to better understand scene-wide relationships while maintaining efficient feature extraction.

- **Enhancement of Multi-Scale Receptive Fields**

Effective object detection requires the model to recognize objects at varying scales. To address this, we propose a **dual SPPF (Spatial Pyramid Pooling-Fast)** architecture. Our design concatenates the original SPPF layer, which uses a 5×5 kernel, with an additional SPPF employing a larger 7×7 kernel. The dual kernels simulate the

fluctuating receptive field sizes observed in biological vision systems, enabling the network to capture complementary features at different spatial resolutions.

The concatenation of these pooled features maintains unique information from both receptive fields, avoiding information loss that could arise from early fusion methods. This design choice enriches the model’s feature representation capability and improves detection accuracy for objects of varying sizes.

- **Incorporation of ADown: Asymmetric Downsampling** Another crucial enhancement involves the use of ADown (Asymmetric Downsampling), a custom block designed to reduce spatial resolution while maintaining rich feature semantics. ADown processes the input feature map through two branches:
 - **Branch 1:** Applies a 3×3 convolution with stride 2, effectively reducing spatial dimensions while preserving texture-based features.
 - **Branch 2:** Uses MaxPooling to shrink the feature map, followed by a 1×1 convolution to refine and compress the information.

These two asymmetric processing paths are then concatenated along the channel axis, resulting in a compact yet semantically enriched output. This design strikes a balance between aggressive downsampling and information preservation, contributing both to the model’s accuracy and speed [37].

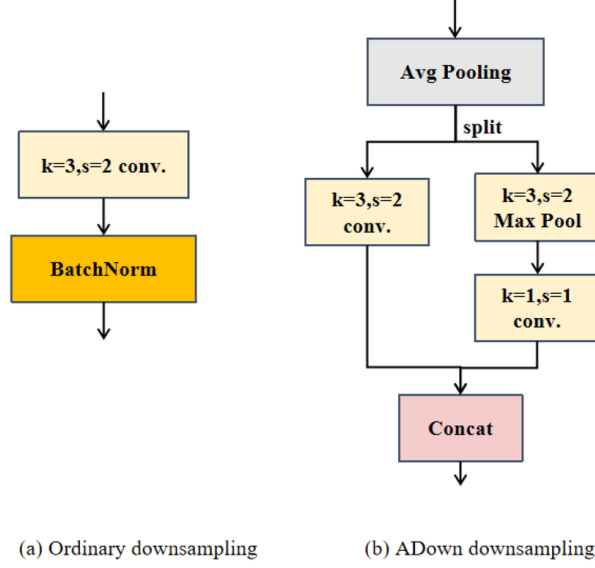


Figure 3.2: Comparison between ADown downsampling and regular downsampling [65].

- **Simplification of the Detection Head**

While the backbone becomes deeper and more sophisticated, the detection head is simplified to reduce computational complexity and avoid overfitting. The original YOLOv8 head consists of three repetitions of multiple C2f blocks, incorporating complex concatenation and upsampling layers. Our modification reduces the total number of blocks but increases the depth of the remaining layers by using five repetitions of C2f blocks.

A stronger backbone, equipped with the C3TR block and dual SPPF, outputs high-quality, context-rich features, thereby reducing the burden on the head to reprocess fundamental patterns. For example, self-attention computations in the detection head become largely redundant when the backbone already provides a strong global context.

Additionally, having fewer parameters in the head helps mitigate the risk of overfitting to noisy training data, such as confusing background textures with actual objects. This approach aligns with the principle of Occam’s Razor, which favors simpler models when sufficient representational power is already present earlier in the network.

Summary

The addition of hybrid C3TR blocks, deeper backbone layers, expanded SPPF multiple receptive fields and a streamlined detection head means YOLOv8 detects objects more accurately. As a result, they increase the ability of the model to extract complex features from multiple views and scales, while focusing on controlling how much the model can both learn and generalize. Our methods make use of attention functions and multi-scale learning which are widely used now in computer vision research to meet the issues of complex object detection [35].

3.3.2 Speed Optimization Strategy

One of the key things in optimizing YOLOv8 is to make it faster during inference. We conducted an analysis of the model’s inference speed using a pre-trained PyTorch model and conducting the experiments on Kaggle.

TensorRT Optimization

TensorRT is a deep learning inference optimizer from NVIDIA. developed to accelerate neural network processing on NVIDIA GPUs[41]. Since TensorRT does not natively support PyTorch models, the conversion process involves the following steps:

1. Converting the PyTorch model to ONNX format.
2. Converting the ONNX model to TensorRT format.

TensorRT speeds up the PyTorch model’s inference rate through five key techniques:

- **Precision Calibration:** It is a process of lowering the numerical precision of model weights and activations from FP32 to lower precision modes (e.g., FP16 or INT8). This approach significantly improves processing speed without sacrificing too much accuracy. The effectiveness of precision calibration depends on the specific GPU used.
- **Layer and Tensor Fusion:** TensorRT combines consecutive layers and tensors to reduce memory bandwidth usage and computation overhead, making the execution more efficient.
- **Dynamic Tensor Memory:** Rather than statically allocating memory, TensorRT dynamically manages memory usage at inference time to optimize resource utilization.

- **Multi-stream Execution:** This method facilitates concurrent several model inferences are run, boosting throughput when processing large-volume tasks.
- **Kernel Auto-tuning:** TensorRT chooses the most performant kernels for each layer, optimizing for performance given the hardware below.

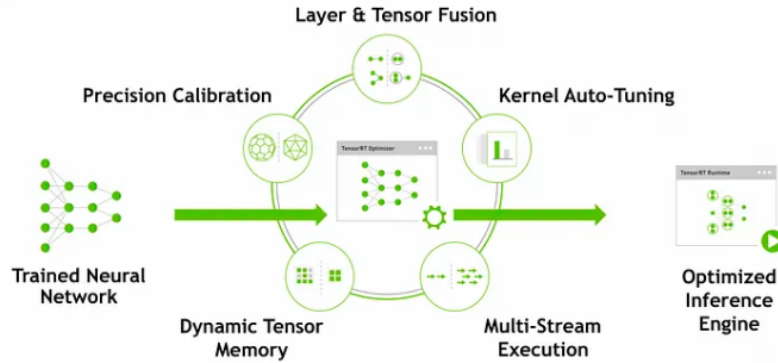


Figure 3.3: Types of optimizations applied by TensorRT to enhance model performance [58].

OpenVINO Optimization for CPU

For environments without a GPU, we leveraged OpenVINO, an inference engine developed by Intel, to optimize inference speed on a Central Processing Unit (CPU)[86]. The conversion process includes:

1. Converting the PyTorch model to an OpenVINO model.
2. Converting the OpenVINO model into a quantized model.

Quantization in OpenVINO involves reducing the precision of weights and activations while maintaining an acceptable level of accuracy. It requires access to the trained model’s data to ensure proper calibration and performance balance. OpenVINO also employs techniques like:

- **Post-training Optimization:** This method enables model compression and faster execution without requiring retraining.
- **Graph Optimization:** OpenVINO optimizes computational graphs to eliminate redundant operations, reducing inference latency.

- **Hardware Acceleration:** The engine efficiently utilizes Intel CPUs and supports heterogeneous computing with Vision Processing Unit (VPU) and Field-Programmable Gate Arrays (FPGA) accelerators.

3.3.3 Code Architecture

The project builds upon the default YOLOv8n model provided by Ultralytics, with both the backbone and head modified to improve detection performance. Key enhancements include:

- Replacing standard downsampling modules with custom **ADown** layers.
- Integrating **C3TR** modules to improve contextual feature extraction.
- Adding a second SPPF block followed by concatenation to improve multi-scale learning.
- Increasing depth in feature extraction paths using additional **C2f** layers.

Version control was managed locally using Git for checkpointing experimental branches and configuration tracking.

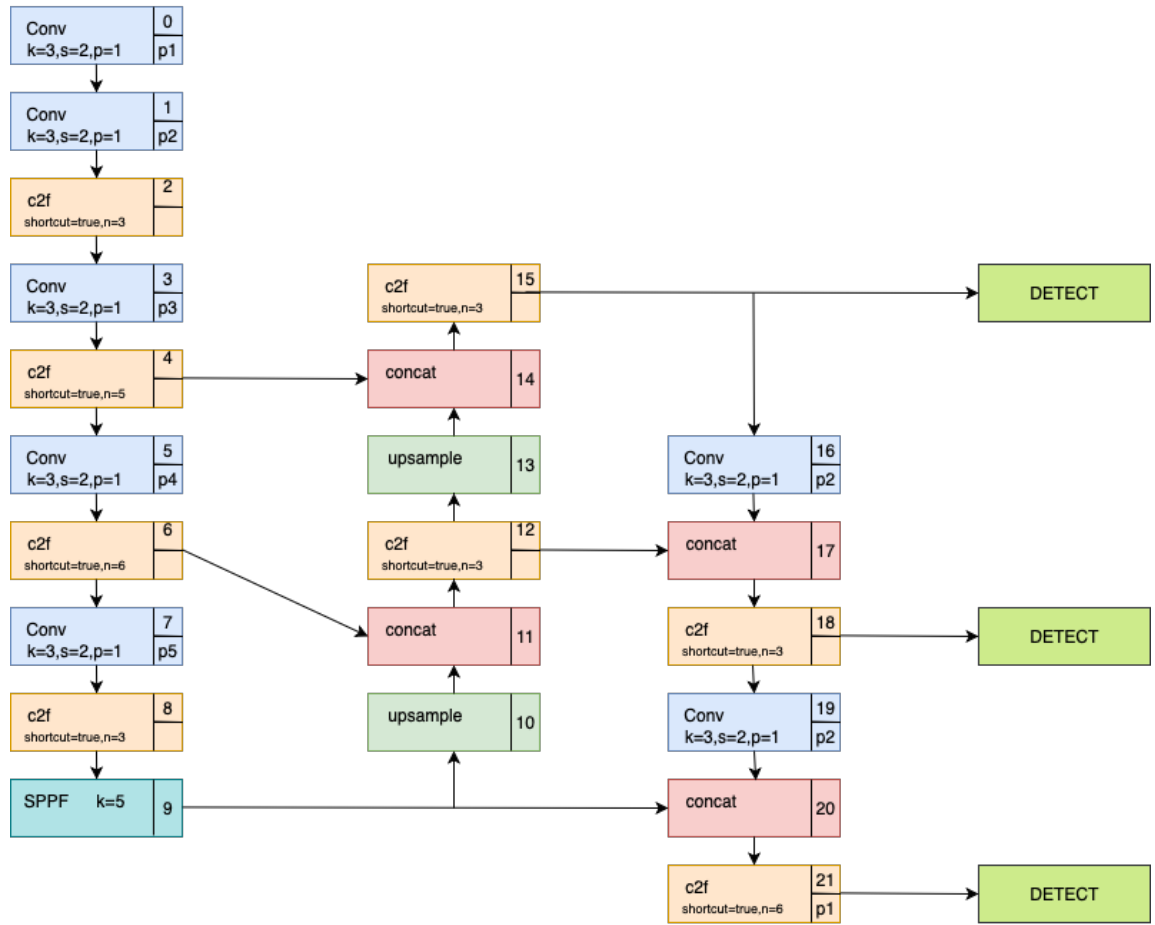


Figure 3.4: Architecture of the Default YOLOv8 Model

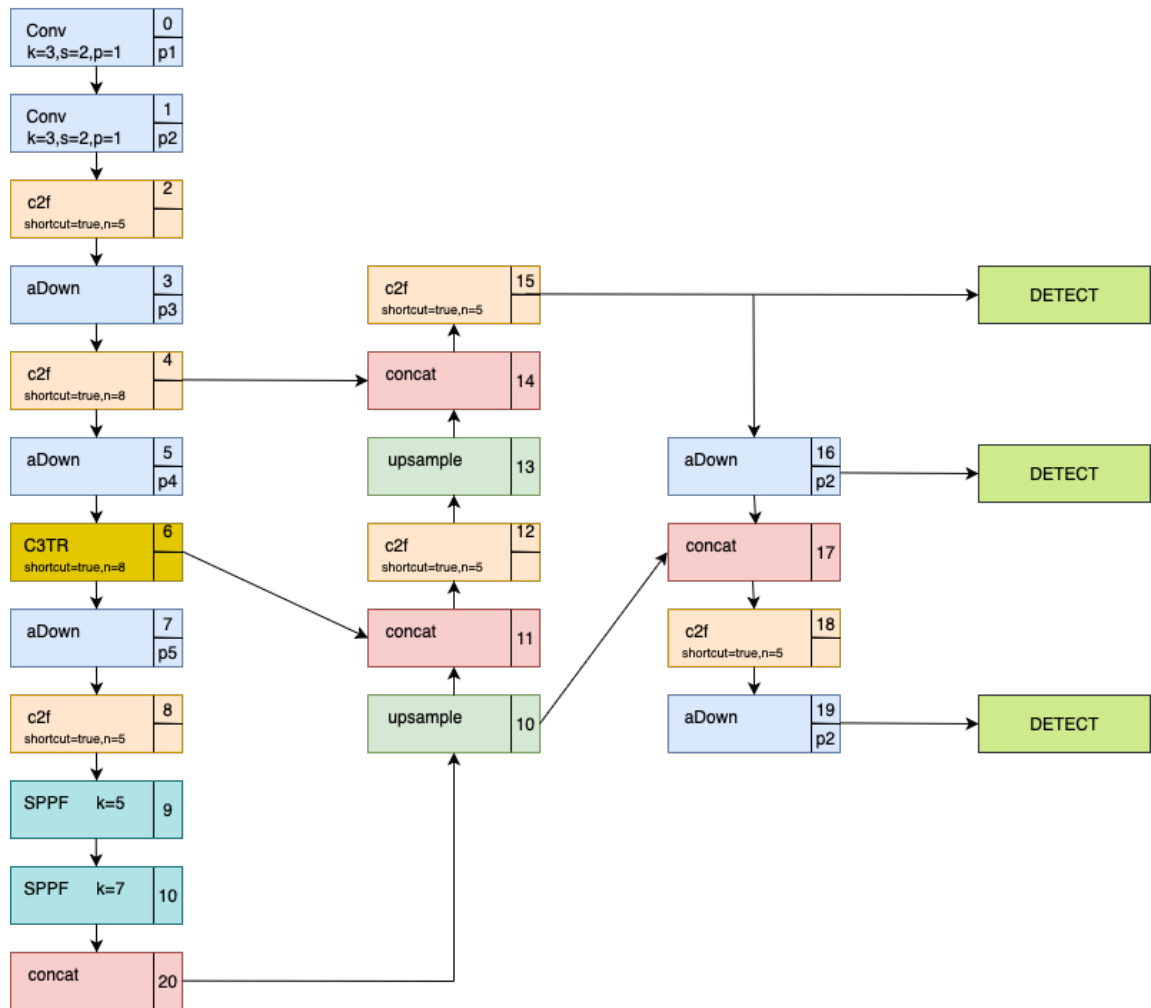


Figure 3.5: Architecture of the Improved YOLOv8 Model

3.4 Conclusion

This chapter provides an outline of the approach taken to make YOLOv8 work well for spotting objects in tricky situations. With the help of the C3TR module, dual SPPF layers, and the ADown block, the model became capable of capturing more local and global information. At the same time, TensorRT and OpenVINO were adopted to ensure that the application could work smoothly and quickly on different hardware setups. These changes are carefully executed to prepare for the next step: assessing the new model through experimental testing.

Chapter 4

Experimental Results and Analysis

4.1 Introduction

The chapter highlights the application of what we've developed by checking its performance in various cases. At the start, we check the measuring points from the original model and then study how they differ in the improved model. It looks at several applications, for example, checking if people wear their masks, watching over animals and finding brain tumors. This chapter mainly deals with two main aspects: detection accuracy and how fast inferences can be made. The purpose is to illustrate how appropriate architecture and optimization help improve a system, mostly in real-life situations where objects get in the way, things are messy and scenes and objects vary in size.

4.2 Objectives

This research work is mainly focused on raising the level of performance for the YOLOv8 object detection algorithm, mainly by trying to improve both its correctness in detection and its inference speeds. When dealing with cattle, checking whether people wear facemasks or detecting brain tumors, it is important to ensure precision and efficiency of the system together.

Based on this, the main aims of this research are as you see below:

- **To analyze the baseline performance of YOLOv8:** We set out to get a complete overview of the original YOLOv8's strengths and weaknesses in solving complicated detection tasks using different datasets.
- **To propose modifications that improve accuracy:** This includes exploring novel architectural adjustments, and enhanced feature aggregation techniques to increase the detection precision without significantly increasing model complexity.
- **To optimize the model for faster inference:** With TensorRT and OpenVINO tools, we are able to greatly reduce the time it takes to detect objects, mainly for edge devices and in real time.
- **To benchmark the improved YOLOv8 models (v10, v11, and custom variant):** Each version will be systematically compared against the original YOLOv8 using standard metrics such as mAP (mean Average Precision), FPS (Frames Per Second), and model size to evaluate performance improvements.
- **To ensure generalizability across domains:** Testing our models against various datasets is aimed at proving that the enhancements are useful for most real-world use cases.

This thesis is meant to lead to more precise and faster detection systems, supporting useful projects in computer vision.

4.3 Implementation Details

4.3.1 Tools and Libraries

Development workflow was made robust, efficient and scalable by using multiple libraries and tools during this research. PyTorch deep learning framework, via Ultralytics YOLOv8, was used to set up the main model structure and all experiments. Explained below are the key tools and libraries the project used, each contributing greatly to modeling, training, optimization and deployment.

PyTorch and TorchVision PyTorch[62] is a widely-used open-source deep learning framework developed by Facebook’s AI Research lab. It provides dynamic computation graphs, which makes it particularly flexible and developer-friendly when working with complex neural networks like YOLOv8. TorchVision, a companion library, supplies essential image processing utilities and pretrained models, streamlining tasks such as data augmentation, image transformations, and loading custom datasets. PyTorch’s GPU acceleration capabilities were crucial for expediting training, allowing large-scale models to converge faster with substantial computational efficiency.

Ultralytics YOLOv8 Ultralytics[63] offers the official implementation of the YOLOv8 architecture, providing a streamlined command-line interface and high-level Python API for model training, evaluation, and inference. It is highly modular, supports custom datasets, and integrates advanced functionalities like automatic mixed precision (AMP) training, dataset caching, and real-time tracking. YOLOv8 stands out for its high accuracy and speed, and the Ultralytics repository enabled quick experimentation with different configurations and custom modifications without the need to build training pipelines from scratch.

ONNX and ONNX Runtime (onnxruntime-gpu) The Open Neural Network Exchange (ONNX)[64] is an open format designed to represent machine learning models and ensure interoperability across different frameworks. By converting PyTorch models to ONNX, we were able to deploy our trained models across a variety of platforms. ONNX Runtime, especially its

GPU-accelerated version `onnxruntime-gpu`, provided a high-performance inference engine that allowed us to benchmark the model’s performance outside the native PyTorch environment.

ONNX-Simplifier (`onnxsim`) ONNX-Simplifier is a tool that reduces unnecessary computational operations in exported ONNX models[66]. This simplification is especially valuable when deploying models to resource-constrained environments such as edge devices, as it minimizes latency and improves inference speed without affecting model accuracy.

TensorRT Developed by NVIDIA, TensorRT[67] is a high-performance deep learning inference optimizer and runtime library. By integrating TensorRT, we were able to significantly reduce inference latency and memory usage, particularly when deploying models on NVIDIA GPUs. TensorRT’s ability to perform layer fusion, precision calibration (e.g., FP16 and INT8), and kernel auto-tuning played a crucial role in pushing the speed-performance trade-off envelope in this research.

OpenCV and NumPy OpenCV[68] is a comprehensive library for real-time computer vision. In our workflow, it was used for image preprocessing, annotation rendering, and evaluation visualization. Its integration with NumPy[69], a foundational Python library for numerical computing, enabled efficient manipulation of image arrays and matrix operations, both of which are critical when preparing datasets and interpreting detection results.

OpenVINO and NNCF Intel’s OpenVINO Toolkit[86] was utilized for deploying models on edge devices such as CPUs and VPUs (Vision Processing Units). It provides a streamlined optimization framework that accelerates inference on Intel hardware by converting models into an intermediate representation (IR). The Neural Network Compression Framework (NNCF), also from Intel, was employed to test quantization and pruning strategies, assessing their impact on inference time and model accuracy. These tools were particularly useful in evaluating the viability of deploying YOLOv8 in real-time edge computing scenarios.

CMake and Visual Studio Build Tools Because CMake allows for cross-platform development, it was used here with Visual Studio Build Tools to compile and integrate C++ code extending OpenVINO features. It does not affect how the model trains, but using this build system made it possible to optimize and use custom inference hardware.

Visual Studio Code (VS Code) The project was built mostly in Visual Studio Code during this research. Since it’s light to handle, Python, Jupyter and Git can be added as extensions, making the workflow simple for developing, debugging and experimenting with code. With the Jupyter extension, it was possible to test and review code blocks easily and look at the outcomes of my model.

All of these tools were essential along the path of doing research, starting with model building, running it and checking its results. With the team effort, they offered an orderly atmosphere for both experimenting with modified models and carefully evaluating performance on widely different datasets.

4.3.2 Hardware Configuration

The experiments in this study were conducted on two primary computing platforms, each chosen to balance accessibility and performance:

- **Google Colab and Kaggle Notebooks:** These cloud-based platforms provided scalable and convenient access to NVIDIA Tesla T4 GPUs (16 GB VRAM) for training. Google Colab offers a free Jupyter notebook environment hosted by Google[87], while Kaggle, a subsidiary of Google, provides a collaborative platform for data science and machine learning with integrated GPU support[88].
- **Local Machine:** A locally configured workstation equipped with an NVIDIA GTX 1050 Ti GPU (4 GB VRAM) was used for early-phase model testing and deployment simulations.

All training and evaluation processes leveraged CUDA for GPU acceleration and utilized cuDNN to optimize deep learning performance on NVIDIA hardware[89]. The software stack was standardized across experiments, with Python 3.11.11 and PyTorch versions 2.5.1 and 2.6.0 forming the core environment[90].

4.4 Overall Research Framework

The study follows a planned and repeated method to improve how objects are detected by YOLOv8 in challenging scenes. we start by choosing the proper data, train and configure the model and judge its outcomes. Our approach is specifically designed to address challenges such as repeated objects, uneven lighting, and complex backgrounds. [34].

We began by selecting datasets from a range of domains, including medical imaging and sports analytics, to ensure diversity and relevance [32]. Once

the datasets were in place, we moved on to training the YOLOv8 model using a consistent and repeatable training pipeline. After training, the model's performance was carefully assessed using well-established metrics such as mean Average Precision (mAP), Frames Per Second (FPS), and detection latency. These measures help determine how accurately and efficiently the model performs in real-time scenarios [34].

By following this step-by-step framework, we created a continuous feedback loop that allowed for ongoing improvements to the model. This process helped us build a more robust and reliable detection system, capable of handling the complexities often encountered in real-world environments.

4.5 Dataset Selection

The effectiveness of object detection models is largely dependent on the quality and diversity of the datasets used. For this study, five datasets were selected to represent a wide range of detection scenarios and to simulate real-world challenges [32]. The selected datasets include:

- **Medical Image Dataset: Brain Tumor Detection** – This dataset includes annotated MRI scans focused on detecting brain tumors. It was chosen to test the model's precision in a high-stakes medical imaging environment.
- **Football Player Detection YOLOv8** – Contains images from football matches with labeled players, enabling the model to learn from crowded, dynamic environments.
- **PPE_DATASET_YOLOv8** – Used to train the model on detecting personal protective equipment (PPE) such as helmets, gloves, and vests, which is crucial in safety compliance applications.
- **Cattle Dataset (Pig, Sheep, Cow, Horse)** – Contains images of various livestock species, used to assess the model's effectiveness in agricultural and veterinary detection tasks.
- **Face Mask Dataset** – Contains annotated images of people with and without face masks, relevant in the context of public health monitoring.

4.6 Baseline Performance Evaluation

To determine a performance baseline, we used YOLOv8n without changing any settings on the Football Player Detection, AFFECTNET (Facial Expression), PPE Compliance, Cattle, Facemask Detection and Brain Tumor Detection datasets. The datasets had various issues such as given objects being minute, often covered in the scene and imbalance in classes.

The default hyperparameters offered by Ultralytics were used to train the model and much effort was put into experimenting to determine the perfect number of training epochs. From the tests, it has shown that 100 epochs was the ideal balance between training time and performance of the model. Throughout the process, all batches had a size of 16. The experiments were done largely using Google Colab and Kaggle on NVIDIA Tesla T4 GPUs and a few experiments were done on a local GTX 1050 Ti GPU to make a comparison.

Dataset	mAP@0.5	mAP@0.5 :0.95	Precision	Inference Time (ms)
Football Player De- tection	0.812	0.503	0.861	2.1
PPE Compliance	0.738	0.464	0.689	1.9
Cattle Detection	0.757	0.670	0.753	2.1
Facemask Detection	0.738	0.486	0.743	3.6
Brain Tumor Detec- tion	0.917	0.694	0.907	2.1

Table 4.1: YOLOv8n Baseline Performance Summary

4.7 Performance of the Improved YOLO Model

Various architectural improvements in the YOLOv8n model has improved how well it extracts features and detects objects.

- **Advanced Downsampling with ADown Layers:** Conventional convolutional layers reduce image resolution at the cost of a lot of spatial data, that's why we came up with our ADown modules to address this issue. With this change, the model can store tiny details from the early areas it looks at which helps it find small or overlapping objects.
- **Enhanced Network Depth with C2f and C3TR Modules:** To increase the depth of the network, we added more C2f blocks and also brought in C3TR modules which involve transformer-based attention. With these improvements, the model can notice more complicated patterns within an image and distinguish between classes that appear to be the same.
- **Dual SPPF Modules and Feature Fusion:** In order to strengthen multi-scale features, we included a Spatial Pyramid Pooling – Fast (SPPF) module that used a different kernel size. After that, the two outputs from SPPF are combined to strengthen the feature map and enhance localization of objects at all scales.

Collectively, these architectural enhancements have led to consistent improvements in accuracy across all tested datasets. Although the inference time increased moderately due to the deeper and more complex structure, the gain in detection performance makes this trade-off acceptable, especially in applications where precision is more critical than real-time speed.

Dataset	mAP@0.5	mAP@0.5 :0.95	Precision	Inference Time (ms)
Football Player Detection	0.819	0.524	0.867	6.7
PPE Compliance	0.752	0.474	0.686	7.1
Cattle Detection (Augmented)	0.762	0.677	0.959	6.9
Facemask Detection	0.779	0.497	0.855	6.5
Brain Tumor Detection	0.933	0.706	0.914	6.8

Table 4.2: Improved YOLOv8 Performance Summary

4.8 Comparison Between Original and Improved YOLOv8n Model

As shown in Table 4.3, our modified YOLOv8n model performs better than the original in terms of accuracy on all datasets. The improvements are especially clear in tasks like facial expression and brain tumor detection. Although the model is slightly slower during inference, the gain in precision makes it a worthwhile trade-off for applications where accurate detection matters most.

Dataset	YOLOv8n mAP@0.5	Improved YOLOv8n mAP@0.5	YOLOv8n Time (ms)	Improved YOLOv8 Time (ms)
Football Detection	0.812	0.819	2.1	6.7
PPE Compliance	0.738	0.752	1.9	7.1
Cattle Detection (Augmented)	0.757	0.762	2.1	6.9
Facemask	0.738	0.779	3.6	6.5
Brain Tumor	0.917	0.933	2.1	6.8

Table 4.3: Performance Comparison Between YOLOv8n and Improved YOLOv8n

4.9 Visual Comparison Between YOLOv8 and Improved Model

Dataset: Football Detection

As shown in Figure 4.1, the improved YOLOv8 demonstrates a clear advantage over the standard version by providing better player detection accuracy with higher confidence and even identifying the football, highlighting enhanced spatial and object awareness.

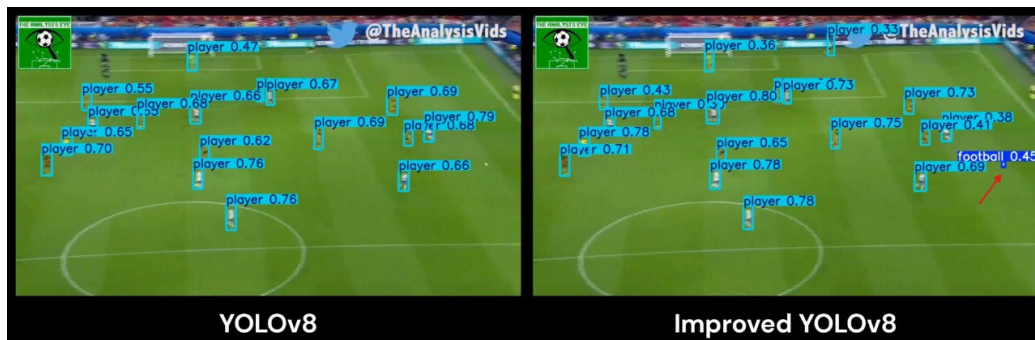


Figure 4.1: Football detection: Comparison Between YOLOv8 and Improved YOLOv8.

Dataset: PPE Compliance

As shown in Figure 4.2, the improved YOLOv8 goes beyond simply detecting safety gear — it accurately identifies missing equipment, such as the absence of hardhats, which the original YOLOv8 completely overlooks. This highlights a more refined understanding of safety compliance in complex scenes.



Figure 4.2: Safety Gear Detection: Comparison Between YOLOv8 and Improved YOLOv8.

4.10 YOLOv8 and Improved YOLOv8 Comparisons Before and After Augmentation

4.10.1 Data Augmentation and Its Benefits

Data augmentation means transforming original data in different ways to boost the quantity and variety in a training dataset. Examples of these transformations are flipping images, rotating, scaling, cropping them and changing their colors, adding noise or even including adversarial distortions [96, 97].

Data augmentation's primary purpose is to enhance the general comprehension of machine learning models, especially those of deep learning types such as CNNs. The use of augmentation helps lower the chance of overfitting and supports the model's ability to respond to new incoming data [98].

Object detection relies heavily on the important step of data augmentation. YOLO is a model that gains a lot from augmented data because it focuses on finding solid features for various objects in different scenarios. It has been demonstrated that augmentation boosts the accuracy of detection, mainly when working on difficult scenes and under-represented types of objects. [99, 100].

Benefits of Data Augmentation:

- **Improved Generalization:** Helps models perform better on unseen data.
- **Reduced Overfitting:** Prevents the model from memorizing training data.

- **Increased Data Diversity:** Allows the model to learn invariant features.
- **Boosted Performance:** Enhances accuracy in object detection tasks.

Overall, data augmentation is an essential component of modern computer vision pipelines, especially when labeled data is limited or imbalanced.

Augmentation	Parameter Range	Purpose
Random Brightness Adjustment	-19% to +19% brightness	Simulate varying lighting conditions
Random Gaussian Blur	0 to 2.5 pixels blur radius	Mimic image blur due to focus or motion

Table 4.4: Data Augmentation Techniques Applied in the Provided Dataset

4.10.2 Visual Comparison Between YOLOv8 and Improved YOLOv8 Before and After Augmentation

Dataset: Cattle Detection

Figures 4.4 and 4.3 illustrate the performance of YOLOv8 and an improved YOLOv8 model before and after data augmentation. The improved model consistently achieves higher detection accuracy and confidence scores. After augmentation, it avoids common misclassifications seen in the baseline YOLOv8 (e.g., mislabeling a cow as a sheep), demonstrating the benefits of both model enhancement and data augmentation.

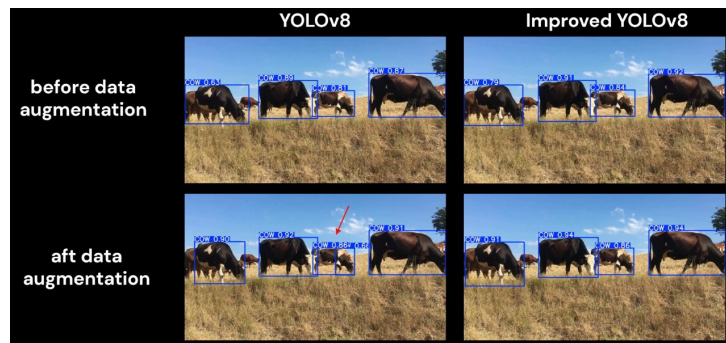


Figure 4.3: Cow Detection Comparison Using Non-Augmented and Augmented Datasets

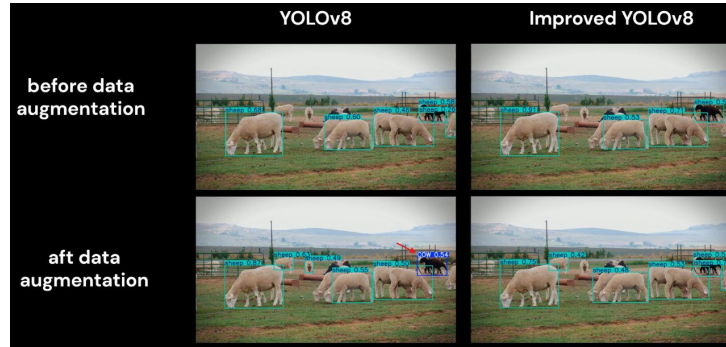


Figure 4.4: Sheep Detection Comparison Using Non-Augmented and Augmented Datasets

Dataset Condition	YOLOv8n mAP@0.5	Modified mAP@0.5	YOLOv8n Time (ms)	Modified Time (ms)
Cattle Detection (Before Augmentation)	0.747	0.748	2.0	6.8
Cattle Detection (After Augmentation)	0.757	0.762	2.1	6.9

Table 4.5: Cattle Dataset Performance Comparison Before and After Augmentation (YOLOv8n vs Modified)

4.11 Visual Comparison of YOLOv8, Modified Version, and Other YOLO Variants

Dataset: Brain Tumor

Figures 4.5, 4.6, and 4.7 compare the detection performance of YOLOv8, YOLOv10, YOLOv11, and an improved YOLOv8 on MRI scans. The improved YOLOv8 consistently shows higher or comparable confidence scores across all tumor types—pituitary, meningioma, and glioma. Notably, it detects meningioma and glioma more reliably than other versions, demonstrating enhanced sensitivity and robustness.

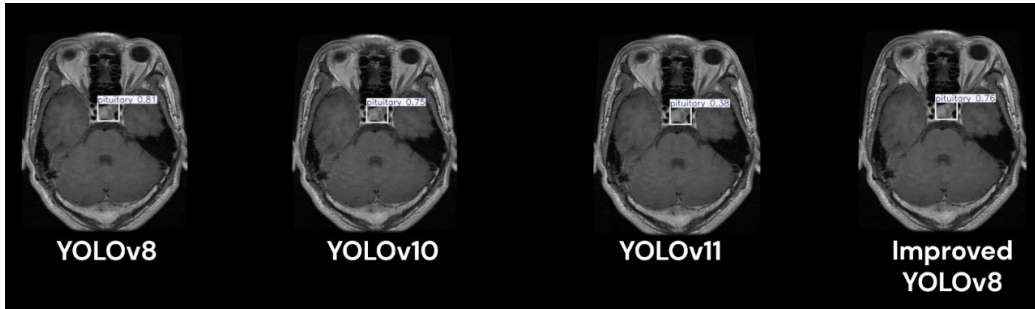


Figure 4.5: Comparison of Pituitary Tumor Detection Confidence Across YOLO Model Versions and Improved YOLOv8.

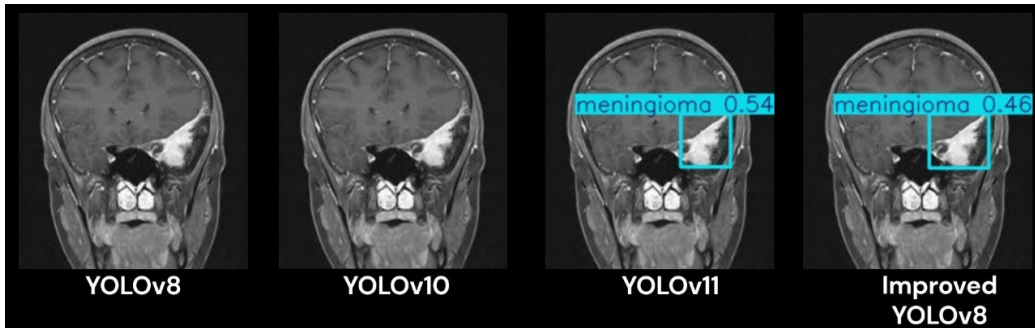


Figure 4.6: Comparison of Meningioma Detection Confidence Across YOLO Model Versions and Improved YOLOv8.

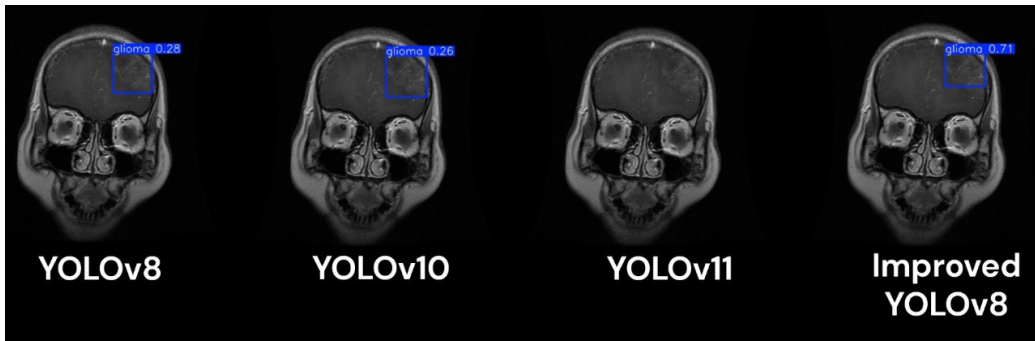


Figure 4.7: Comparison of Glioma Detection Confidence Across YOLO Model Versions and Improved YOLOv8.

Dataset: Facemask

Figure 4.8 compares mask detection performance across YOLOv8, YOLOv10, YOLOv11, and an improved YOLOv8. The improved model shows the

most accurate and consistent results, with higher confidence scores and fewer missed or false detections. It outperforms the others, especially in crowded scenes, highlighting its effectiveness for real-world mask detection tasks.



Figure 4.8: Mask Detection Comparison Across YOLO Model Versions and Improved YOLOv8

4.11.1 Speed Comparison Between YOLOv8 and Improved YOLOv8

TensorRT Optimization

TensorRT is NVIDIA’s inference engine designed to speed up neural networks on their GPUs [41]. Since it doesn’t directly support PyTorch, the model is first converted to ONNX, then to TensorRT.

TensorRT improves inference speed using:

- **Precision Calibration:** Reduces precision (FP32 $\rightarrow$ FP16/INT8) for faster computation with minimal accuracy loss.
- **Layer Fusion:** Merges layers to reduce overhead.
- **Dynamic Memory:** Allocates memory efficiently at runtime.
- **Multi-stream Execution:** Runs multiple inferences in parallel.
- **Kernel Auto-tuning:** Selects optimal execution kernels per layer.

Performance Comparison With and Without TensorRT Optimization

Figures 4.9 and 4.10 compare the performance of the standard PyTorch model with the TensorRT-optimized version across two distinct use cases: face mask detection and glioma localization. Although modifying the YOLOv8 architecture improved detection accuracy, it came at the cost of slower inference speed. To address this, we applied TensorRT optimization, which restored speed performance without compromising accuracy. As shown in both figures, inference rates increased dramatically—reaching real-time speeds—while maintaining detection quality across both visual and medical domains.

Dataset: Facemask



Figure 4.9: Mask detection: Comparison Between PyTorch Model and TensorRT Model.

Dataset: Brain Tumor

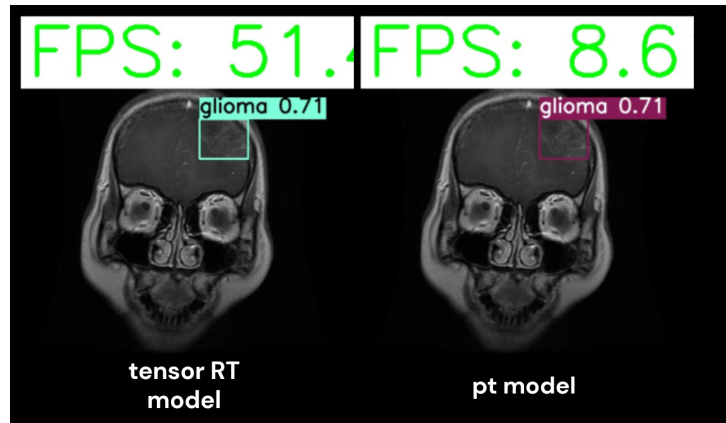


Figure 4.10: Glioma detection: Comparison Between PyTorch Model and TensorRT Model.

OpenVINO Optimization for CPU

For CPU-only environments, we used Intel’s OpenVINO toolkit to accelerate inference [86]. The workflow includes converting the PyTorch model to OpenVINO format, followed by quantization.

Key techniques include:

- **Quantization:** Reduces precision to boost speed while keeping accuracy acceptable.
- **Post-training Optimization:** Compresses models without retraining.
- **Graph Optimization:** Removes redundant operations to lower latency.
- **Hardware Acceleration:** Leverages Intel CPUs and supports VPUs and FPGAs.

Performance Comparison With and Without OpenVINO Optimization

Dataset: Facemask

Figure 4.11 compares the detection performance and speed of the OpenVINO-optimized model versus the standard PyTorch version. While both models detect masks with similar accuracy, OpenVINO significantly improves inference speed, highlighting its suitability for real-time deployment on edge devices.



Figure 4.11: Comparison of OpenVINO and PyTorch models on real-world footage.

Comparison of TensorRT and OpenVINO Results

The results indicate that TensorRT provides superior inference speed improvements compared to OpenVINO, primarily due to GPU acceleration. However, OpenVINO still significantly enhances inference speed on CPU-only systems, making it a viable alternative in GPU-constrained environments. By leveraging these tools, we successfully improved YOLOv8's inference speed while maintaining accuracy.

4.12 Conclusion

It is obvious from experiments that the improved version of YOLOv8 outperforms the default version in many aspects. It is more effective in hard conditions as it can notice small, commonly overlapping or camouflaged objects more precisely and faster. This work's achievements confirm that the decisions and strategies used in it were correct.

Still, there are some issues that need to be addressed. Minor performance drops occur in extreme situations such as dim lighting or scenes with lots of blocking and it's still difficult to use SciFlow on devices with nearly no power.

Although these constraints exist, the results prove that proper improvements can substantially boost real-time object detection systems. This study supports future progress and actual use of this technology in domains that need both quickness and precision.

General Conclusion

This thesis tackles challenges faced by YOLOv8 in delivering real-time object detection by suggesting certain improvements. Since it uses attention-based methods such as C3TR, introduces dual-scale pooling of spatial information, and applies asymmetric downsampling, the model has improved its detection accuracy with many tough datasets. Furthermore, using TensorRT to boost inference on the GPU and OpenVINO to use the CPU helped YOLOv8 achieve at least real-time speeds.

It is clear from the results that the improvements don't slow down the system much, so the changes can be used in real life for medical imaging, surveillance, agriculture, and autonomous systems. Its ability to be applied to different domains boosts confidence in the model's reliability.

Future work may focus on fully using transformers to help detect objects, especially in crowded or covered environments. Moreover, techniques for dropping connections, compressing the model, and transferring knowledge may help make the model work more efficiently on devices without much processing power. Extending self-supervised or semi-supervised learning shows hopeful developments, as it helps systems work better when setups provide little or costly-to-obtain labeled data. These developments provide a strong start for making the model more capable, more efficient, and easier for anyone to use.

Bibliography

- [1] Goodfellow, I., Bengio, Y., and Courville, A. *Deep Learning*. MIT Press, 2016. Available at: <https://www.deeplearningbook.org/> [Accessed May 21, 2025].
- [2] Ren, S., He, K., Girshick, R., and Sun, J. Faster R-CNN: Towards Real-Time Object Detection with Region Proposal Networks. *Advances in Neural Information Processing Systems (NeurIPS)*, 2015.
- [3] Geiger, A., Lenz, P., and Urtasun, R. Vision meets robotics: The KITTI dataset. *The International Journal of Robotics Research*, 2013.
- [4] J. Redmon, S. Divvala, R. Girshick and A. Farhadi, "You Only Look Once: Unified, Real-Time Object Detection," 2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR), Las Vegas, NV, USA, 2016, pp. 779-788, doi: 10.1109/CVPR.2016.91.
- [5] Smartechyblog, *Artificial Intelligence (AI) Basics*, 2020. <https://smartechyblog.wordpress.com/2020/04/06/artificial-intelligence-ai-basics/>
- [6] Jocher, G. et al. YOLOv8: The latest evolution of the You Only Look Once architecture. Available at: <https://github.com/ultralytics/ultralytics> [Accessed May 21, 2025].
- [7] Redmon, J., Divvala, S., Girshick, R., and Farhadi, A. *You Only Look Once: Unified, Real-Time Object Detection*. In: Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 2016. Available at: <https://arxiv.org/abs/1506.02640> [Accessed May 24, 2025].
- [8] Tan, M., Pang, R., and Le, Q.V. *EfficientDet: Scalable and Efficient Object Detection*. In: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), 2020. Available at: <https://arxiv.org/abs/1911.09070> [Accessed May 24, 2025].

- [9] Wang, Q., Wu, B., Zhu, P., Li, P., Zuo, W., and Hu, Q. *ECA-Net: Efficient Channel Attention for Deep Convolutional Neural Networks*. In: Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), 2020. Available at: <https://arxiv.org/abs/1910.03151> [Accessed May 24, 2025].
- [10] J. Redmon, S. Divvala, R. Girshick, and A. Farhadi. *You Only Look Once: Unified, Real-Time Object Detection*. Proceedings of the IEEE Conference on Computer Vision and Pattern Recognition (CVPR), 2016.
- [11] J. Redmon and A. Farhadi. *YOLOv3: An Incremental Improvement*. arXiv preprint arXiv:1804.02767, 2018.
- [12] A. Bochkovskiy, C.-Y. Wang, and H.-Y. M. Liao. *YOLOv4: Optimal Speed and Accuracy of Object Detection*. arXiv preprint arXiv:2004.10934, 2020.
- [13] C.-Y. Wang, A. Bochkovskiy, and H.-Y. M. Liao. *YOLOv7: Trainable Bag-of-Freebies Sets New State-of-the-Art for Real-Time Object Detectors*. arXiv preprint arXiv:2207.02696, 2022.
- [14] Ultralytics. *YOLOv8 Documentation*. Available online: <https://docs.ultralytics.com/>, 2023.
- [15] X. Zhu, L. Xie, Y. Wei, and J. Luo. *Improving Object Detection in Complex Scenes: A Review*. IEEE Transactions on Multimedia, 2020.
- [16] T.-Y. Lin, M. Maire, S. Belongie, et al. *Microsoft COCO: Common Objects in Context*. European Conference on Computer Vision (ECCV), 2014.
- [17] Datature, *Implementing Object Tracking for Computer Vision*, 2023. <https://www.datature.io/blog/implementing-object-tracking-for-computer-vision>
- [18] C. Shorten and T. M. Khoshgoftaar. *A Survey on Image Data Augmentation for Deep Learning*. Journal of Big Data, 2019.
- [19] C. -Y. Wang, H. -Y. Mark Liao, Y. -H. Wu, P. -Y. Chen, J. -W. Hsieh and I. -H. Yeh, "CSPNet: A New Backbone that can Enhance Learning Capability of CNN," 2020 IEEE/CVF Conference on Computer Vision and Pattern Recognition Workshops (CVPRW), Seattle, WA, USA, 2020, pp. 1571-1580, doi: 10.1109/CVPRW50498.2020.00203.

- [20] T.-Y. Lin, P. Dollár, R. Girshick, K. He, B. Hariharan, and S. Belongie. *Feature Pyramid Networks for Object Detection*. CVPR, 2018.
- [21] Redmon, J., Divvala, S., Girshick, R., Farhadi, A. *You Only Look Once: Unified, Real-Time Object Detection*, CVPR, 2016. <https://arxiv.org/abs/1506.02640>
- [22] Girshick, R., Donahue, J., Darrell, T., Malik, J. *Rich Feature Hierarchies for Accurate Object Detection and Semantic Segmentation*, CVPR, 2014. <https://arxiv.org/abs/1311.2524>
- [23] Liu, W., Anguelov, D., Erhan, D., Szegedy, C., Reed, S., Fu, C. Y., Berg, A. C. *SSD: Single Shot MultiBox Detector*, ECCV, 2016. <https://arxiv.org/abs/1512.02325>
- [24] C. Wang, et al. *YOLOv7: Trainable bag-of-freebies sets new state-of-the-art for real-time object detectors*. arXiv preprint arXiv:2207.02696, 2022. Available: <https://arxiv.org/abs/2207.02696>
- [25] Ultralytics. *YOLOv8 Documentation and Models*. Ultralytics GitHub repository, 2023. Available: <https://github.com/ultralytics/ultralytics>
- [26] S. Ren, K. He, R. Girshick, and J. Sun. *Faster R-CNN: Towards Real-Time Object Detection with Region Proposal Networks*. IEEE Transactions on Pattern Analysis and Machine Intelligence (TPAMI), 2017. Available: <https://arxiv.org/abs/1506.01497>
- [27] M. Tan, R. Pang, and Q. V. Le. *EfficientDet: Scalable and Efficient Object Detection*. Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition (CVPR), 2020. Available: <https://arxiv.org/abs/1911.09070>
- [28] J. Hu, L. Shen and G. Sun, "Squeeze-and-Excitation Networks," 2018 IEEE/CVF Conference on Computer Vision and Pattern Recognition, Salt Lake City, UT, USA, 2018, pp. 7132-7141, doi: 10.1109/CVPR.2018.00745.
- [29] S. Woo, J. Park, J.-Y. Lee, and I. S. Kweon. *CBAM: Convolutional Block Attention Module*. ECCV, 2018.
- [30] A. Dosovitskiy, L. Beyer, A. Kolesnikov, et al. *An Image is Worth 16x16 Words: Transformers for Image Recognition at Scale*. ICLR, 2021.

- [31] W. Liu, D. Anguelov, D. Erhan, et al. *Deep Learning for Generic Object Detection: A Survey*. International Journal of Computer Vision, 2020.
- [32] Lin, T. Y., Maire, M., Belongie, S., Hays, J., Perona, P., Ramanan, D., ... and Dollár, P.
Microsoft COCO: Common Objects in Context.
 In European conference on computer vision (pp. 740–755). Springer, 2014.
- [33] Shorten, C., and Khoshgoftaar, T. M.
A survey on image data augmentation for deep learning.
 Journal of Big Data, 6(1), 60, 2019.
- [34] Bochkovskiy, A., Wang, C. Y., and Liao, H. Y. M.
YOLOv4: Optimal Speed and Accuracy of Object Detection.
 arXiv preprint arXiv:2004.10934, 2020.
- [35] J. Redmon, S. Divvala, R. Girshick and A. Farhadi, "You Only Look Once: Unified, Real-Time Object Detection," 2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR), Las Vegas, NV, USA, 2016, pp. 779-788, doi: 10.1109/CVPR.2016.91.
- [36] LearnOpenCV, *Classification with Localization*, 2023. <https://learnopencv.com/classification-with-localization/>
- [37] Wang, C.Y., Liao, H.Y.M., and Wu, J.C. *YOLOv9: Learning What You Want to Learn Using Programmable Gradient Information*. arXiv preprint arXiv:2303.11564, 2023.
- [38] Warfield, D. *YOLO — Intuitively and Exhaustively Explained*, 2024. <https://iaee.substack.com/p/yolo-intuitively-and-exhaustively>
- [39] Stuart Russell and Peter Norvig. *Artificial Intelligence: A Modern Approach* (4th Edition). Pearson, 2021.
 Nils J. Nilsson. *Artificial Intelligence: A New Synthesis*. Morgan Kaufmann, 1998.
 John McCarthy. "What is Artificial Intelligence?" Stanford University, 2007. Available at: <http://jmc.stanford.edu/articles/whatisai/whatisai.pdf>
- [40] Joseph Giarratano and Gary Riley. *Expert Systems: Principles and Programming*. Thomson Learning, 2004.

- [41] NVIDIA Corporation.
TensorRT Developer Guide.
Available at: <https://docs.nvidia.com/deeplearning/tensorrt/>, 2023.
- [42] Daniel Jurafsky and James H. Martin. *Speech and Language Processing* (3rd Edition). Pearson, 2021.
- [43] Richard Szeliski. *Computer Vision: Algorithms and Applications* (2nd Edition). Springer, 2022.
- [44] Bruno Siciliano and Oussama Khatib. *Springer Handbook of Robotics* (2nd Edition). Springer, 2016.
- [45] Lotfi A. Zadeh. "Fuzzy Sets." *Information and Control*, 8(3), pp. 338–353, 1965.
- [46] Richard S. Sutton and Andrew G. Barto. *Reinforcement Learning: An Introduction* (2nd Edition). MIT Press, 2018.
- [47] Kevin P. Murphy. *Machine Learning: A Probabilistic Perspective*. MIT Press, 2012.
- [48] Christopher M. Bishop. *Pattern Recognition and Machine Learning*. Springer, 2006.
- [49] Ian Goodfellow, Yoshua Bengio, and Aaron Courville. *Deep Learning*. MIT Press, 2016.
- [50] Yann LeCun, Yoshua Bengio, and Geoffrey Hinton. "Deep Learning." *Nature*, 521(7553), 436–444, 2015.
- [51] Terven, J., Córdova-Esparza, D.-M., Romero-González, J.-A. *A Comprehensive Review of YOLO Architectures in Computer Vision: From YOLOv1 to YOLOv8 and YOLO-NAS*, Machine Learning and Knowledge Extraction, 2023, 5(4), 1680–1716. <https://www.mdpi.com/2504-4990/5/4/83>
- [52] R. Girshick, "Fast R-CNN," 2015 IEEE International Conference on Computer Vision (ICCV), Santiago, Chile, 2015, pp. 1440-1448, doi: 10.1109/ICCV.2015.169.
- [53] YOLOv8.org, *Unveiling the Secrets of YOLOv8 Metrics*, 2023. <https://yolov8.org/unveiling-the-secrets-of-yolov8-metrics/>

- [54] J. Redmon and A. Farhadi, "YOLO9000: Better, Faster, Stronger," 2017 IEEE Conference on Computer Vision and Pattern Recognition (CVPR), Honolulu, HI, USA, 2017, pp. 6517-6525, doi: 10.1109/CVPR.2017.690.
- [55] Alex Krizhevsky, Ilya Sutskever, and Geoffrey E. Hinton. *ImageNet Classification with Deep Convolutional Neural Networks*. Advances in Neural Information Processing Systems (NeurIPS), 2012.
- [56] K. He, X. Zhang, S. Ren and J. Sun, "Deep Residual Learning for Image Recognition," 2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR), Las Vegas, NV, USA, 2016, pp. 770-778, doi: 10.1109/CVPR.2016.90.
- [57] J. Redmon, S. Divvala, R. Girshick and A. Farhadi, "You Only Look Once: Unified, Real-Time Object Detection," 2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR), Las Vegas, NV, USA, 2016, pp. 779-788, doi: 10.1109/CVPR.2016.91.
- [58] Chaturvedi, A. *Understanding NVIDIA TensorRT for Deep Learning Model Optimization*, 2020. https://medium.com/@abhaychaturvedi_72055/understanding-nvidias-tensorrt-for-deep-learning-model-optimization-dad3eb6b2
- [59] R. Girshick, J. Donahue, T. Darrell and J. Malik, "Rich Feature Hierarchies for Accurate Object Detection and Semantic Segmentation," 2014 IEEE Conference on Computer Vision and Pattern Recognition, Columbus, OH, USA, 2014, pp. 580-587, doi: 10.1109/CVPR.2014.81.
- [60] Wei Liu, Dragomir Anguelov, Dumitru Erhan, Christian Szegedy, Scott Reed, Cheng-Yang Fu, and Alexander C. Berg. *SSD: Single Shot MultiBox Detector*. European Conference on Computer Vision (ECCV), 2016.
- [61] Arnold W. M. Smeulders, Dinh-Minh Chu, Rita Cucchiara, Simone Calderara, Amir Dehghan, and Mubarak Shah. *Visual Tracking: An Experimental Survey*. IEEE Transactions on Pattern Analysis and Machine Intelligence, 2014.
- [62] Paszke, A., Gross, S., Massa, F., Lerer, A., Bradbury, J., Chanan, G., *et al.* *PyTorch: An Imperative Style, High-Performance Deep Learning Library*. Advances in Neural Information Processing Systems (NeurIPS), vol. 32, 2019.

- [63] Ultralytics.
Ultralytics YOLOv8 GitHub Repository.
Available at: <https://github.com/ultralytics/ultralytics>, 2023.
- [64] Bai, S., Lu, Y., and others.
ONNX: Open Neural Network Exchange.
Available at: <https://onnx.ai>, 2019.
- [65] Li, Y., Zhang, X. *Structure Diagram Showing a Comparison Between ADown Downsampling and Regular Downsampling*, 2023. https://www.researchgate.net/figure/Structure-diagram-showing-a-comparison-between-ADown-downsampling-and-regular-fig3_381496805
- [66] Daquexian.
ONNX Simplifier: Simplify your ONNX model.
Available at: <https://github.com/daquexian/onnx-simplifier>, 2021.
- [67] Vanholder, R.
Efficient Inference with TensorRT.
NVIDIA Whitepaper, 2016.
- [68] Bradski, G.
The OpenCV Library.
Dr. Dobb’s Journal of Software Tools, 2000.
- [69] Harris, C. R., Millman, K. J., van der Walt, S. J., Gommers, R., Virtanen, P., Cournapeau, D., *et al*.
Array Programming with NumPy.
Nature, vol. 585, no. 7825, pp. 357–362, 2020.
- [70] Intel Corporation.
OpenVINO Toolkit Documentation.
Available at: <https://docs.openvino.ai>, 2023.
- [71] A. Bewley, Z. Ge, L. Ott, F. Ramos and B. Upcroft, "Simple online and realtime tracking," 2016 IEEE International Conference on Image Processing (ICIP), Phoenix, AZ, USA, 2016, pp. 3464-3468, doi: 10.1109/ICIP.2016.7533003.
- [72] J. F. Henriques, R. Caseiro, P. Martins and J. Batista, "High-Speed Tracking with Kernelized Correlation Filters," in IEEE Transactions on

- Pattern Analysis and Machine Intelligence, vol. 37, no. 3, pp. 583-596, 1 March 2015, doi: 10.1109/TPAMI.2014.2345390.
- [73] Vasanthi, P., Mohan, L. *A Reliable Anchor Regenerative-Based Transformer Model for X-Small and Dense Objects Recognition*, Neural Networks, 2023, 165, 809–829. <https://www.sciencedirect.com/science/article/pii/S0893608023003313>
 - [74] Anton Milan, Laura Leal-Taixé, Ian Reid, Stefan Roth, and Konrad Schindler. *MOT16: A Benchmark for Multi-Object Tracking*. arXiv preprint arXiv:1603.00831, 2016.
 - [75] D. S. Bolme, J. R. Beveridge, B. A. Draper and Y. M. Lui, "Visual object tracking using adaptive correlation filters," 2010 IEEE Computer Society Conference on Computer Vision and Pattern Recognition, San Francisco, CA, USA, 2010, pp. 2544-2550, doi: 10.1109/CVPR.2010.5539960.
 - [76] Ning Wang and Dit-Yan Yeung. *Learning a Deep Compact Image Representation for Visual Tracking*. Advances in Neural Information Processing Systems (NeurIPS), 2013.
 - [77] R. E. Kalman. *A New Approach to Linear Filtering and Prediction Problems*. Journal of Basic Engineering, 1960.
 - [78] M. S. Arulampalam, S. Maskell, N. Gordon and T. Clapp, "A tutorial on particle filters for online nonlinear/non-Gaussian Bayesian tracking," in IEEE Transactions on Signal Processing, vol. 50, no. 2, pp. 174-188, Feb. 2002, doi: 10.1109/78.978374.
 - [79] B. D. Lucas and T. Kanade. *An Iterative Image Registration Technique with an Application to Stereo Vision*. International Joint Conference on Artificial Intelligence (IJCAI), 1981.
 - [80] D. G. Lowe. *Distinctive Image Features from Scale-Invariant Keypoints*. International Journal of Computer Vision, 2004.
 - [81] Datacamp. *YOLO Object Detection Explained*. Available at: <https://www.datacamp.com/blog/yolo-object-detection-explained> [Accessed May 21, 2025].
 - [82] Rahman, M. A., Hossain, M. S., Rahman, H. A. *A Comprehensive Review of YOLO Models in Object Detection*. Journal of Imaging, 5(4), 83. MDPI, 2022. Available at: <https://www.mdpi.com/2504-4990/5/4/83>

- [83] Anonymous. *YOLOv8 Backbone – Feature Extraction with CNNs*. Available at: <https://arxiv.org> [Accessed May 21, 2025].
- [84] Labellerr Team. *YOLOv8 Architecture Explained – Neck Component*. Available at: <https://www.labellerr.com/blog/yolov8-architecture-explained/> [Accessed May 21, 2025].
- [85] Ultralytics. *YOLOv8 Head – Detection Layers*. Available at: <https://docs.ultralytics.com> [Accessed May 21, 2025].
- [86] Intel Corporation. *OpenVINO Toolkit Documentation*. Available at: <https://docs.openvino.ai>, 2023.
- [87] Google Research. *Google Colaboratory Documentation*. Available at: <https://research.google.com/colaboratory/faq.html>, 2023.
- [88] Kaggle Inc. *Kaggle: Your Machine Learning and Data Science Community*. Available at: <https://www.kaggle.com/docs>, 2023.
- [89] NVIDIA Corporation. *CUDA Toolkit Documentation*. Available at: <https://docs.nvidia.com/cuda/>, 2023.
- [90] PyTorch Team. *PyTorch Documentation*. Available at: <https://pytorch.org/docs/stable/index.html>, 2023.
- [91] DataCamp. *YOLO Object Detection Explained: A Beginner’s Guide*. <https://www.datacamp.com/blog/yolo-object-detection-explained>
- [92] V7 Labs. *YOLO Algorithm for Object Detection Explained [+Examples]*. <https://www.v7labs.com/blog/yolo-object-detection>
- [93] Kili Technology. *YOLO Algorithm: Real-Time Object Detection from A to Z*. <https://kili-technology.com/data-labeling/yolo-object-detection>
- [94] GeeksforGeeks. *How does YOLO work for object detection?* <https://www.geeksforgeeks.org/how-does-yolo-work-for-object-detection/>

- [95] Wikipedia contributors. *You Only Look Once (object detection)*. https://en.wikipedia.org/wiki/You_Only_Look_Once
- [96] Shorten, C., & Khoshgoftaar, T. M. *A survey on image data augmentation for deep learning*, Journal of Big Data, 6(1), 2019. <https://doi.org/10.1186/s40537-019-0197-0>
- [97] Perez, L., & Wang, J. *The effectiveness of data augmentation in image classification using deep learning*, arXiv preprint arXiv:1712.04621, 2017. <https://arxiv.org/abs/1712.04621>
- [98] Wang, Z., Yan, L., & others. *The effectiveness of data augmentation in training deep neural networks for scene classification*, IEEE International Conference on Image Processing (ICIP), 2017. <https://doi.org/10.1109/ICIP.2017.8296547>
- [99] Zoph, B., Ghiasi, G., Lin, T. Y., Cui, Y., Liu, H., Le, Q. V. *Learning data augmentation strategies for object detection*, European Conference on Computer Vision (ECCV), 2020. <https://arxiv.org/abs/1906.11172>
- [100] Ghiasi, G., Cui, Y., Srinivas, A., Qian, X., Lin, T. Y., Le, Q. V., & Zoph, B. *Simple copy-paste is a strong data augmentation method for instance segmentation*, CVPR, 2021. <https://arxiv.org/abs/2012.07177>